

Visual CeTZ

Guide visuel de `cetz:0.5.2` pour Typst 0.15.1

🔧 FERGOUS Abdelhak

Chaque construction est montrée par son code et son rendu, côte à côte.
Dans l'esprit de [VisualTikZ](#) pour TikZ.

Comment lire ce guide. Le code de gauche est exactement le contenu d'un `cetz.canvas({ ... })` après `import cetz.draw: *`. Le rendu de droite en est la sortie réelle, compilée avec CeTZ 0.5.2. Les exemples sont autonomes : copiez-collez, ça marche.

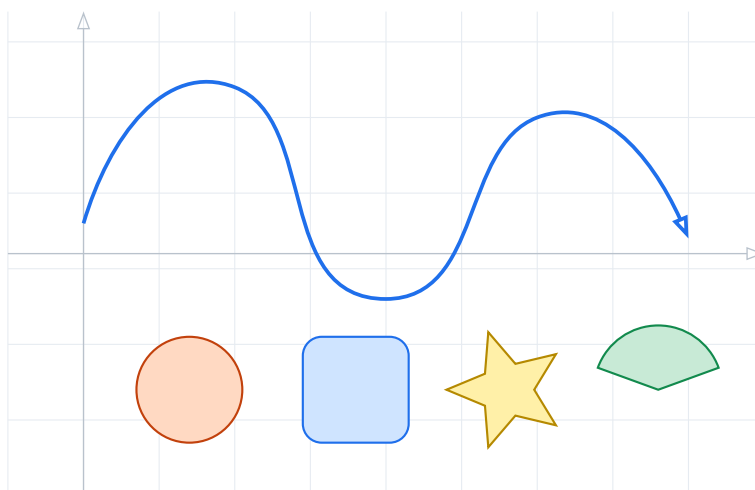


Table des matières

1	Prise en main	4
1.1	Installation & squelette minimal	4
1.2	Le conteneur canvas	5
1.3	Anatomie d'un appel	5
1.4	Convention de ce guide pour les imports	5
1.5	Panorama des fonctions	5
2	Coordonnées	6
2.1	Tableau récapitulatif	6
2.2	Cartésien et polaire	6
2.3	Relatif et point précédent	6
2.4	Interpolation le long d'un segment	7
2.5	Perpendiculaires et projection	7
2.6	Barycentrique	7
2.7	Tangente	7
2.8	Coordonnée fonctionnelle	7
3	Les formes de base	7
3.1	line — segments et lignes brisées	7
3.2	rect — rectangles	8
3.3	circle et circle-through	8
3.4	arc et arc-through	8
3.5	polygon et n-star	9
3.6	grid	9
3.7	content — insérer du contenu Typst	9
3.8	Courbes : bezier, bezier-through, catmull, hobby	10
3.9	merge-path et compound-path	10
3.10	svg-path	10
3.11	rect-around	10
4	Styles	10
4.1	Trois niveaux de style	10
4.2	Raccourcis fill et stroke	11
4.3	Clés de style principales	11
4.4	Remplissages avancés	11
5	Marques (flèches)	11
5.1	Mnémoniques	11
5.2	Noms de formes	12
5.3	Position et combinaisons	12
5.4	Taille, remplissage, inclinaison	12
5.5	Marques sur les courbes	12
5.6	mark comme élément autonome	13
6	Ancre	13
6.1	Ancre de boussole	13
6.2	Ancre de chemin	13
6.3	L'argument anchor: — placer l'élément par son ancre	13
6.4	Créer ses propres ancrs	13
6.5	Parcourir les ancrs : for-each-anchor	14
6.6	copy-anchors	14
7	Groupe, calques et intersections	14
7.1	group	14
7.2	scope — isoler sans créer d'élément	14
7.3	hide et floating	14
7.4	on-layer	14
7.5	intersections	15
7.6	get-ctx / set-ctx	15
8	Transformations	15
8.1	translate	15
8.2	rotate	15
8.3	scale	15
8.4	set-origin et move-to	15
8.5	set-viewport	15
8.6	Matrices : set-transform / transform	16
9	Dessin 3D	16
9.1	ortho — projection isométrique	16
9.2	perspective	16

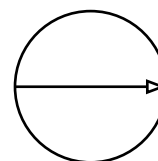
9.3 Plans de travail on-xy / on-xz / on-zy	16
10 Opérations booléennes	16
11 Décorations	17
11.1 zigzag, coil, wave, square	17
11.2 Accolades : brace et flat-brace	17
12 Angles	18
13 Arbres	18
14 Palettes	19
15 Recettes	20
15.1 Diagramme de boîtes et flèches	20
15.2 Étiqueter une flèche	20
15.3 Repère et courbe de fonction	20
15.4 Cotation	20
15.5 Hachures d'une zone (via boolean)	20
15.6 Graphe avec nœuds circulaires	21
15.7 Placer un dessin dans le texte	21
15.8 Réutiliser un motif	21
16 Vecteurs et matrices	21
16.1 <code>cetz.vector</code>	21
16.2 <code>cetz.matrix</code>	22
17 Étendre CeTZ	22
17.1 Marques personnalisées : <code>register-mark</code>	22
17.2 Systèmes de coordonnées personnalisés	23
17.3 Éléments réutilisables	23
17.4 Interroger le contexte	23
18 L'écosystème : <code>cetz-plot</code>	24
18.1 Courbes de fonctions	24
18.2 Styles d'axes	24
18.3 Remplissages et zones	24
18.4 Échelles et formats de graduation	25
18.5 Barres d'erreur, aires, contours	25
19 Diagrammes : <code>chart</code>	25
19.1 Barres et colonnes	25
19.2 Camembert	26
19.3 Radar	26
19.4 Boîte à moustaches	26
20 Diagrammes de Venn : <code>cetz-venn</code>	26
21 SmartArt : processus et cycles	27
21.1 Processus linéaire	27
21.2 Cycle	27
22 Recettes avancées	28
22.1 Automate à états finis	28
22.2 Réseau de neurones	28
22.3 Diagramme de Gantt	28
22.4 Cercle trigonométrique	29
22.5 Empilement 3D avec tri de profondeur	29
22.6 Carte de chaleur	29
22.7 Chronogramme (signaux logiques)	29
22.8 Molécule / graphe chimique	30
22.9 Papier millimétré / motif répété	30
23 Showcase : détails souvent ignorés	30
23.1 <code>grid</code> : <code>shift</code> , <code>help-lines</code> , pas asymétrique	30
23.2 <code>hobby</code> : <code>omega</code> et tensions par segment	30
23.3 <code>catmull</code> : tension et fermeture	31
23.4 <code>content</code> : texte le long d'une ligne, <code>wrap</code> , <code>auto-scale</code>	31
23.5 <code>arc</code> : <code>update-position</code> et ancrs	31
23.6 <code>line</code> : ancrs numérotées des sommets	31
23.7 <code>mark</code> : <code>offset</code> , <code>sep</code> , <code>shorten-to</code> , <code>transform-shape</code>	32
23.8 <code>intersections</code> : tri des points	32
23.9 <code>floating</code> : ignorer un élément dans les bornes	32
23.10 <code>hide</code> : calculé mais non dessiné	32
23.11 <code>set-origin</code> vs <code>translate</code>	32
23.12 <code>on-layer</code> : ordre de peinture indépendant du code	33
23.13 Ancres par distance ou par ratio	33
23.14 <code>set-viewport</code> : inverser un axe	33

23.15	Dégradés suivant la forme	33
23.16	Motifs (tiling)	33
23.17	Styles imbriqués et . . d'un dictionnaire	33
24	Galerie	33
24.1	Le théorème de Pythagore	34
24.2	Coupe géologique / couches empilées	34
24.3	Engrenage paramétrique	34
24.4	Rosace (courbe polaire)	35
24.5	Spirale d'Archimède et suite de Fibonacci	35
24.6	Diagramme de Voronoï (approché)	35
24.7	Carte mentale (arbre stylisé)	36
24.8	Sphère filaire (méridiens et parallèles)	36
24.9	Chemin de fer / diagramme syntaxique	36
25	Couverture complémentaire	36
25.1	canvas : x, y, z, background, stroke	37
25.2	Décorations : le style au complet	37
25.3	brace : accolade effilée	37
25.4	Arbres : champs du nœud et alignement	38
25.5	plot : types de ligne	38
25.6	plot : hypographe, épigraphe, remplissage	38
25.7	plot.annotate : dessiner dans les coordonnées du tracé	39
25.8	plot.add-bar : barres dans un tracé	39
25.9	plot.add-contour : lignes de niveau	39
25.10	plot.add-violin : distribution	39
25.11	plot : légende et ancres personnalisées	40
25.12	plot.formats : formats de graduation	40
25.13	chart.piechart : étiquettes et éclatement	40
25.14	chart.columnchart : mode empilé	40
25.15	smartart : options de mise en forme	40
26	Géométrie euclidienne	41
26.1	Les points de base	41
26.2	Cercle circonscrit et médiatrices	41
26.3	Orthocentre par intersection des hauteurs	42
26.4	Cercle inscrit : un barycentre bien choisi	42
26.5	La droite d'Euler	43
26.6	Le cercle des neuf points	44
26.7	Théorème de Thalès	45
26.8	Théorème de l'angle inscrit	45
26.9	Tangentes à un cercle	45
26.10	Deux cercles : points communs et axe radical	46
26.11	Puissance d'un point	46
26.12	Triangle médian et triangle orthique	46
26.13	Droite de Simson	46
26.14	Théorème de Ptolémée	47
26.15	Polygones réguliers et pentagramme	47
26.16	Théorème de Napoléon	48
26.17	Spirale de Théodore	48
26.18	Coniques : ellipse, parabole, hyperbole	49
27	Aide-mémoire	50

1 Prise en main

1.1 Installation & squelette minimal

```
#import "@preview/cetz:0.5.2"
#cetx.canvas({
  import cetx.draw: *
  circle((0, 0), radius: 1)
  line((-1, 0), (1, 0), mark: (end: ">"))
})
```



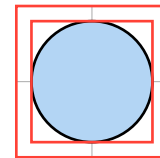
Règle d'or Un canvas prend un **bloc de code** (`{...}`), pas du contenu markup. Les fonctions de dessin s'importent depuis `cetz.draw`. Le tout est évalué paresseusement : chaque fonction renvoie une liste d'éléments, elle ne dessine rien immédiatement.

1.2 Le conteneur canvas

`canvas(body, length: 1cm, debug: false, background: none, padding: none)`

- `length` — longueur d'une unité du repère (1 unité = 1 cm par défaut).
- `debug` — affiche le repère, les bornes et l'origine.
- `background` — couleur/pattern de fond.
- `padding` — marge autour du dessin (nombre, tableau ou dictionnaire).

```
grid((0,0), (2,2), stroke: gray + .3pt)
circle((1,1), radius: .8, fill: blue.lighten(70%))
```



```
// length: 0.5cm -> même dessin, deux fois plus petit
circle((1,1), radius: .8, fill: blue.lighten(70%))
```



1.3 Anatomie d'un appel

```
//      coordonnées      nom      style
line((0,0), (3,1), name: "l", stroke: 2pt + orange)
// le nom permet de réutiliser les ancres
circle("l.end", radius: .2, fill: red)
content("l.start", anchor: "east", padding: .1)[A]
```



Trois familles d'arguments reviennent partout :

Argument	Rôle
positionnels	coordonnées ou contenu de la forme
name:	nom de l'élément, pour interroger ses ancres plus tard
anchor:	ancre de l'élément qui sera placée sur la coordonnée donnée
..style	paires clé-valeur de style (fill, stroke, radius, mark, ...)

1.4 Convention de ce guide pour les imports

Les exemples supposent toujours l'import de base, sous-entendu :

```
#import "@preview/cetz:0.5.2"
#cezt.canvas({
  import cetz.draw: *
  ...
})
```

Dès qu'un exemple utilise un **autre** paquet, sa ligne d'import figure en commentaire sur la première ligne du code :

```
// #import "@preview/cetz-plot:0.1.4": plot
plot.plot(size: (4, 2), { plot.add(domain: (-2, 2), x => x * x) })
```

1.5 Panorama des fonctions

Catégorie	Fonctions
Formes	line rect circle circle-through arc arc-through polygon n-star grid content bezier bezier-through catmull hobby merge-path compound-path svg-path rect-around mark
Style	set-style fill stroke register-mark
Groupes	group scope anchor copy-anchors for-each-anchor hide floating on-layer intersections get-ctx set-ctx
Transformations	translate rotate scale set-origin move-to set-transform transform set-viewport
Booléens	boolean (union, intersection, difference, xor)
3D	ortho perspective on-xy on-xz on-zy
Bibliothèques	cetz.tree cetz.angle cetz.decorations cetz.palette

2 Coordonnées

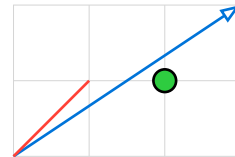
CeTZ résout automatiquement le **système de coordonnées** d'après la forme de la valeur que vous écrivez. Il n'y a rien à déclarer : (1, 2) est cartésien, (30deg, 2) est polaire, "a.north" est une ancre, etc.

2.1 Tableau récapitulatif

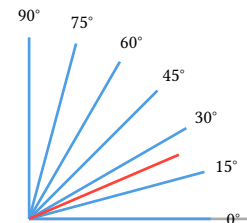
Système	Écriture	Sens
XYZ	(x, y) / (x, y, z) (x: 1, y: 2)	cartésien; unités du canvas ou longueurs ((1cm, 2cm))
Polaire	(30deg, 2) (angle: 30deg, radius: 2)	angle + rayon (rayon scalaire ou (rx, ry))
Ancre	"nom.ancre" / "nom" (name: "n", anchor: "north")	point d'un élément nommé
Barycentrique	(bary: (a: 1, b: 1, c: 2))	moyenne pondérée de points nommés
Relatif	(rel: (1, 0)) (rel: (1,0), to: "a.east")	décalage depuis le point précédent (ou to)
Interpolation	(a, 50%, b) / (a, .5, b) (a, 50%, 20deg, b)	le long du segment a-b, avec rotation optionnelle
Perpendiculaire	(a, "- ", b) / (a, " -", b)	intersection des perpendiculaires
Projection	(p, "_ _", a, b)	projeté orthogonal de p sur la droite ab
Tangente	(element: "c", point: p, solution: 1)	tangente à une forme depuis un point
Fonction	(v => ..., a, b)	calcul libre sur les vecteurs résolus
Précédente	()	le dernier point utilisé

2.2 Cartésien et polaire

```
grid((0,0), (3,2), stroke: gray.lighten(50%) + .3pt)
line((0,0), (3,2), stroke: blue, mark: (end: ">"))
line((0,0), (1cm, 1cm), stroke: red) //
longueurs absolues
circle((x: 2, y: 1), radius: .15, fill: green)
```

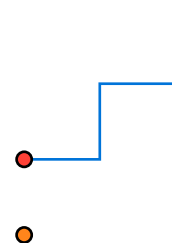


```
line((0,0), (3,0), stroke: gray)
for a in range(0, 91, step: 15) {
  line((0,0), (a * 1deg, 2.4), stroke: blue.lighten(30%))
  content((a * 1deg, 2.7), text(7pt) [#a°])
}
// rayon elliptique : (angle, (rx, ry))
line((0,0), (45deg, (2.8, 1.2)), stroke: red + 1pt)
```



2.3 Relatif et point précédent

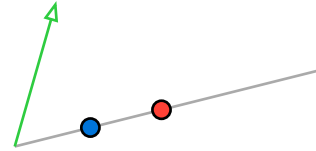
```
// (rel:) : décalage depuis le point précédent
line((0,0), (rel: (1,0)), (rel: (0,1)),
      (rel: (1,0)), (rel: (0,1)), stroke: blue + 1pt)
// (rel:, to:) : décalage depuis une ancre donnée
circle((0,0), radius: .1, fill: red, name: "o")
circle((rel: (0,-1), to: "o"), radius: .1, fill: orange)
```



update (rel: (1,0), update: false) calcule le point relatif **sans** déplacer le « point précédent ». Pratique pour poser des étiquettes en marge d'un chemin.

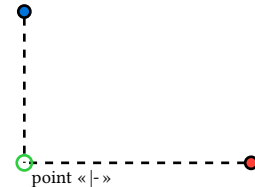
2.4 Interpolation le long d'un segment

```
line((0,0), (4,1), name: "l", stroke: gray)
// ratio -> fraction du segment
circle(((0,0), 25%, (4,1)), radius: .12, fill: blue)
// nombre -> distance absolue en unités
circle(((0,0), 2, (4,1)), radius: .12, fill: red)
// avec angle -> tourne autour du premier point
line((0,0), ((0,0), 2, 60deg, (4,1)), stroke: green, mark: (end: ">"))
```

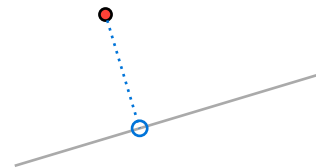


2.5 Perpendiculaires et projection

```
circle((0,2), radius: .08, fill: blue, name: "a")
circle((3,0), radius: .08, fill: red, name: "b")
line("a", ("a", "|-", "b"), stroke: (dash: "dashed"))
line("b", ("a", "|-", "b"), stroke: (dash: "dashed"))
circle(("a", "|-", "b"), radius: .1, stroke: green)
content(("a", "|-", "b"), anchor: "north-west",
padding: .1, text(7pt)[point "|-"])
```

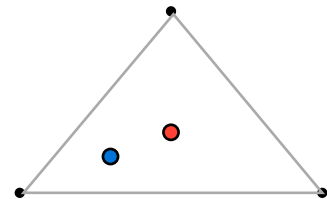


```
line((0,0), (4,1.2), name: "L", stroke: gray + 1pt)
circle((1.2, 2), radius: .08, fill: red, name: "P")
// projeté orthogonal de P sur (0,0)-(4,1.2)
line("P", ("P", "_|_", (0,0), (4,1.2)),
stroke: (paint: blue, dash: "dotted"))
circle(("P", "_|_", (0,0), (4,1.2)), radius: .1, stroke: blue)
```



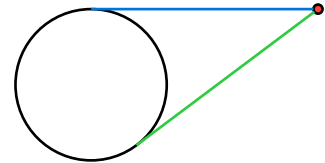
2.6 Barycentrique

```
circle((0,0), radius: .05, name: "A", fill: black)
circle((4,0), radius: .05, name: "B", fill: black)
circle((2,2.4), radius: .05, name: "C", fill: black)
line("A", "B", "C", close: true, stroke: gray)
circle((bary: (A: 1, B: 1, C: 1)), radius: .1, fill: red)
circle((bary: (A: 3, B: 1, C: 1)), radius: .1, fill: blue)
```



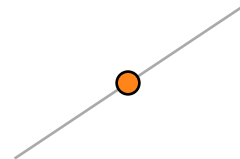
2.7 Tangente

```
circle((0,0), radius: 1, name: "c")
circle((3,1), radius: .06, fill: red, name: "p")
line("p", (element: "c", point: "p", solution: 1), stroke: blue)
line("p", (element: "c", point: "p", solution: 2), stroke: green)
```



2.8 Coordonnée fonctionnelle

```
let mid = (a, b) => vector.scale(vector.add(a, b), .5)
line((0,0), (3,2), stroke: gray)
circle((mid, (0,0), (3,2)), radius: .15, fill: orange)
```

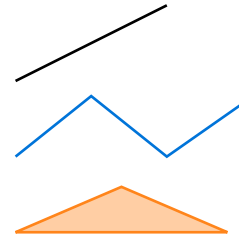


3 Les formes de base

3.1 line — segments et lignes brisées

```
line(..pts, close: false, name: none, ..style)
```

```
line((0,0), (2,1))
line((0,-1), (1,-.2), (2,-1), (3,-.3), stroke: blue)
line((0,-2), (1.4,-1.4), (2.8,-2), close: true,
      fill: orange.lighten(60%), stroke: orange)
```



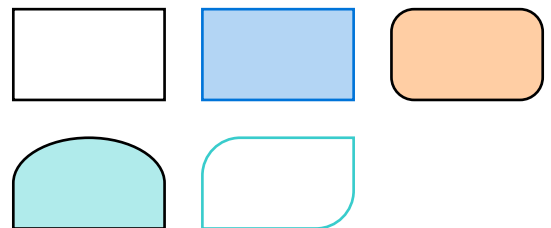
```
// tous les styles de trait Typst sont acceptés
line((0,2), (4,2), stroke: 2pt + blue)
line((0,1.5), (4,1.5), stroke: (paint: red, dash: "dashed"))
line((0,1), (4,1), stroke: (paint: green, dash: "dotted",
  thickness: 1.5pt))
line((0,.5), (4,.5), stroke: (paint: black, cap: "round",
  thickness: 4pt))
```



3.2 rect — rectangles

`rect(a, b, name: none, anchor: none, ..style)`

```
rect((0,0), (2,1.2))
rect((2.5,0), (4.5,1.2), fill: blue.lighten(70%), stroke: blue)
rect((5,0), (7,1.2), radius: .3, fill: orange.lighten(60%))
rect((0,-1.7), (2,-.5), radius: (north: 50%, rest: 0), fill:
  teal.lighten(60%))
rect((2.5,-1.7), (4.5,-.5), radius: (north-west: .5, south-
  east: .5),
  stroke: teal + 1pt)
```

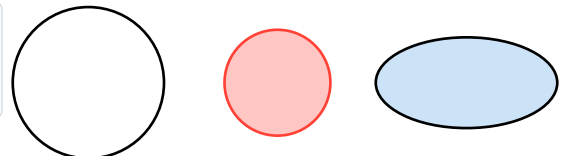


rayon des coins `radius` accepte un nombre, un ratio, un couple (`rx`, `ry`), ou un dictionnaire avec les clés `north` `south` `east` `west` `north-west` `north-east` `south-west` `south-east` `rest`.

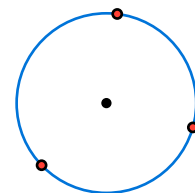
3.3 circle et circle-through

`circle(center, radius: 1, name: none, anchor: none, ..style)`

```
circle((0,0))
circle((2.5,0), radius: .7, fill: red.lighten(70%), stroke: red)
// rayon (rx, ry) -> ellipse
circle((5,0), radius: (1.2, .6), fill: blue.lighten(80%))
```



```
// cercle passant par trois points
circle-through((0,0), (2,.5), (1,2), stroke: blue, name: "c")
for p in ((0,0), (2,.5), (1,2)) { circle(p, radius: .06, fill:
  red) }
circle("c.center", radius: .05, fill: black)
```



3.4 arc et arc-through

`arc(position, start: auto, stop: auto, delta: auto, mode: "OPEN", anchor: none, ..style)`

Exactement **deux** des trois paramètres `start`, `stop`, `delta` doivent être donnés.

```
arc((0,0), start: 0deg, stop: 120deg)
arc((2.5,0), start: 45deg, delta: 180deg, stroke: blue)
arc((5,0), stop: 90deg, delta: 90deg, stroke: red)
```



```
// mode : OPEN (défaut), CLOSE, PIE
arc((0,0), start: 30deg, stop: 150deg, mode: "OPEN", stroke:
black)
arc((3,0), start: 30deg, stop: 150deg, mode: "CLOSE",
fill: blue.lighten(70%), stroke: blue)
arc((6,0), start: 30deg, stop: 150deg, mode: "PIE",
fill: orange.lighten(60%), stroke: orange)
```

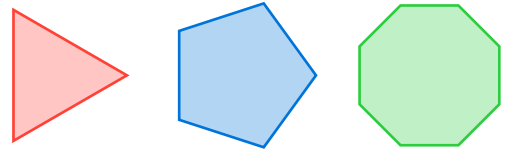


```
// arc passant par trois points
arc-through((0,0), (1,1), (2,0), stroke: 1pt + purple)
```

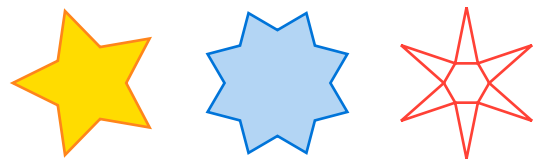


3.5 polygon et n-star

```
polygon((0,0), 3, fill: red.lighten(70%), stroke: red)
polygon((2.5,0), 5, fill: blue.lighten(70%), stroke: blue)
polygon((5,0), 8, angle: 22.5deg, fill: green.lighten(70%),
stroke: green)
```



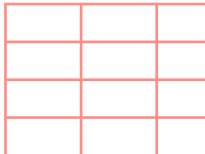
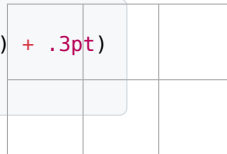
```
n-star((0,0), 5, fill: yellow, stroke: orange)
n-star((2.5,0), 8, inner-radius: 70%, fill: blue.lighten(70%),
stroke: blue)
n-star((5,0), 6, inner-radius: 30%, show-inner: true, stroke:
red)
```



3.6 grid

```
grid(from, to, step: 1, help-lines: false, name: none, ..style)
```

```
grid((0,0), (3,2), stroke: gray + .4pt)
grid((4,0), (7,2), step: .5, stroke: blue.lighten(50%) + .3pt)
grid((8,0), (11,2), step: (x: 1, y: .5), stroke:
red.lighten(40%))
```



3.7 content — insérer du contenu Typst

```
content(position, [body], angle: 0deg, anchor: none, name: none, ..style)
```

```
content(a, b, [body], ..)
```

Le contenu est mis à l'échelle dans le rectangle a-b.

```
content((0,0))[Texte simple]
content((0,-1), text(fill: red)[Coloré])
content((0,-2), angle: 30deg)[Tourné]
content((3,-1), padding: .2, frame: "rect",
fill: yellow.lighten(60%), stroke: orange)[Encadré]
content((3,-2.2), padding: .2, frame: "circle",
fill: blue.lighten(70%))[Bulle]
```

Texte simple

Coloré

Encadré

Tourné

Bulle

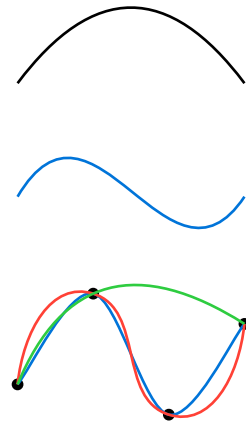
```
// deux coordonnées -> contenu ajusté à la boîte
rect((0,0), (4,1.5), stroke: gray)
content((0,0), (4,1.5))[Contenu redimensionné dans la boîte]
```

Contenu redimensionné dans la boîte

3.8 Courbes : bezier, bezier-through, catmull, hobby

```
// bezier(depart, arrivee, ctrl1, [ctrl2])
bezier((0,0), (3,0), (1.5,2)) //
quadratique
bezier((0,-1.5), (3,-1.5), (1,.2), (2,-3), stroke: blue) //
cubique
```

```
let pts = ((0,0), (1,1.2), (2,-.4), (3,.8))
for p in pts { circle(p, radius: .06, fill: black) }
catmull(..pts, stroke: blue, tension: .5)
hobby(..pts, stroke: red)
bezier-through(pts.at(0), pts.at(1), pts.at(3), stroke: green)
```



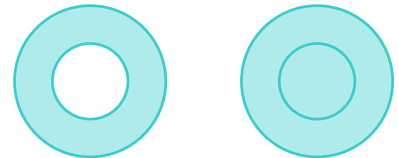
catmull vs hobby catmull (spline de Catmull–Rom, style tension) et hobby (algorithme de John Hobby, style omega) passent tous deux par les points donnés ; hobby produit des courbes plus « esthétiques », plus proches de plot[smooth]/METAFONT.

3.9 merge-path et compound-path

```
// merge-path : recolle plusieurs chemins en un seul chemin
merge-path(fill: blue.lighten(70%), stroke: blue, close: true, {
  line((0,0), (2,0))
  bezier((2,0), (2,2), (3,.5), (3,1.5))
  line((2,2), (0,2))
})
```



```
// compound-path : plusieurs sous-chemins, une seule règle de remplissage
compound-path(fill: teal.lighten(60%), stroke: teal,
  fill-rule: "even-odd", {
    circle((0,0), radius: 1)
    circle((0,0), radius: .5)
  })
compound-path(fill: teal.lighten(60%), stroke: teal,
  fill-rule: "non-zero", {
    circle((3,0), radius: 1)
    circle((3,0), radius: .5)
  })
```



3.10 svg-path

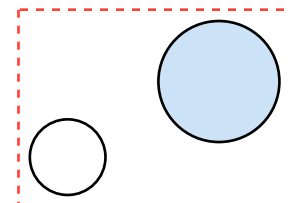
```
svg-path(("m", (0,0)), ("l", (2,0)), ("c", (0,1), (0,0), (-1,1)),
  ("v", -.5), ("h", -1), ("z",),
  fill: orange.lighten(60%), stroke: orange, name: "s")
```



3.11 rect-around

Boîte englobante de coordonnées et/ou d'éléments nommés :

```
circle((0,0), radius: .5, name: "a")
circle((2,1), radius: .8, fill: blue.lighten(80%), name: "b")
rect-around("a", "b", padding: .15,
  stroke: (paint: red, dash: "dashed"))
```



4 Styles

4.1 Trois niveaux de style

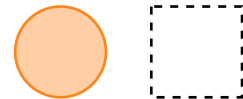
Niveau	Exemple
--------	---------

par élément	<code>circle((0,0), fill: red)</code>
par contexte	<code>set-style(fill: red)</code> — vaut pour la suite du groupe
par racine	<code>set-style(circle: (fill: red))</code> — ne vise que les cercles

```
set-style(stroke: (paint: blue, thickness: 1pt), fill:
blue.lighten(85%))
circle((0,0), radius: .6)
rect((1.2,-.6), (2.4,.6))
// surcharge locale
circle((3.6,0), radius: .6, fill: red.lighten(70%), stroke: red)
```



```
// style par racine : seuls les cercles sont touchés
set-style(circle: (fill: orange.lighten(60%), stroke: orange),
rect: (stroke: (dash: "dashed")))
circle((0,0), radius: .6)
rect((1.2,-.6), (2.4,.6))
```



Héritage La valeur auto d'une clé de style signifie « hérite de l'ancêtre le plus proche ». Ainsi `set-style(stroke: red)` colore aussi les traits des `circle`, dont `circle.stroke` vaut auto par défaut.

4.2 Raccourcis fill et stroke

```
fill(yellow) // = set-style(fill: yellow)
stroke(2pt + red) // = set-style(stroke: 2pt + red)
circle((0,0), radius: .6)
rect((1.4,-.6), (2.6,.6))
```

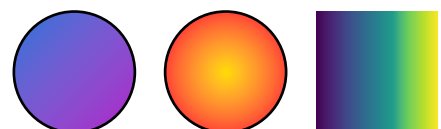


4.3 Clés de style principales

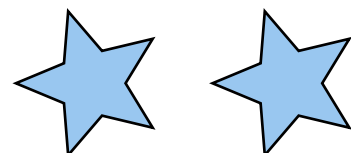
Clé	Défaut	Effet
fill	none	couleur, dégradé ou motif de remplissage
fill-rule	"non-zero"	"non-zero" ou "even-odd"
stroke	1pt noir	trait (type stroke Typst ou dictionnaire)
radius	1	rayon par défaut des cercles/polygones
padding	none	marge (nombre, tableau ou dictionnaire)
mark	voir §5	flèches et symboles de chemin
shorten	"LINEAR"	raccourcissement des béziers pour les marques

4.4 Remplissages avancés

```
circle((0,0), radius: .8,
fill: gradient.linear(blue, purple, angle: 45deg))
circle((2,0), radius: .8,
fill: gradient.radial(yellow, red))
rect((3.2,-.8), (4.8,.8),
fill: gradient.linear(.color.map.viridis), stroke: none)
```



```
// non-zero vs even-odd sur un chemin auto-sécant
compound-path(fill: blue.lighten(60%), fill-rule: "non-zero", {
n-star((0,0), 5, inner-radius: 45%)
})
compound-path(fill: blue.lighten(60%), fill-rule: "even-odd", {
n-star((2.6,0), 5, inner-radius: 45%)
})
```

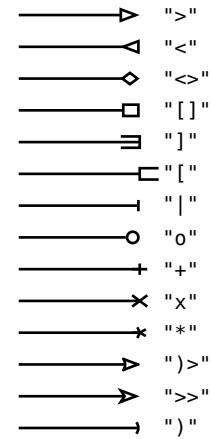


5 Marques (flèches)

5.1 Mnémoniques

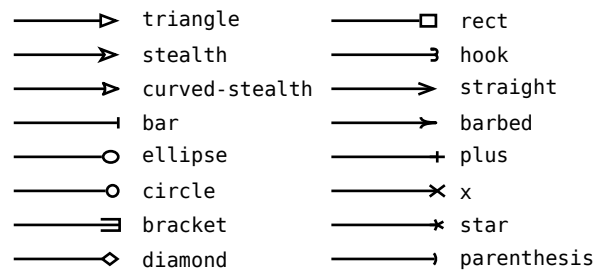
Chaque mnémonique est un raccourci d'une forme de marque :

```
let ms = (">", "<", "<>", "[ ]", "[", "|",
          "o", "+", "x", "*", ">>", ">")
for (i, m) in ms.enumerate() {
  let y = -i * .42
  line((0, y), (1.6, y), mark: (end: m))
  content((1.9, y), anchor: "west", raw("\\" + m + "\""))
}
```



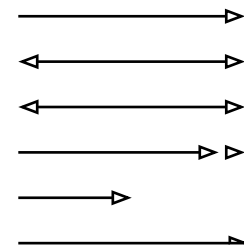
5.2 Noms de formes

```
let ms = ("triangle", "stealth", "curved-stealth", "bar",
          "ellipse", "circle", "bracket", "diamond", "rect", "hook",
          "straight", "barbed", "plus", "x", "star", "parenthesis")
for (i, m) in ms.enumerate() {
  let (cx, cy) = (int(i / 8) * 4.2, calc.rem(i, 8) * -.45)
  line((cx, cy), (cx + 1.4, cy), mark: (end: m))
  content((cx + 1.7, cy), anchor: "west", text(7pt,
  raw(m)))
}
```



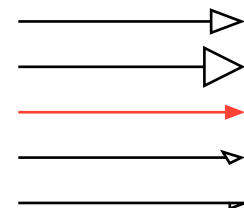
5.3 Position et combinaisons

```
line((0,3), (3,3), mark: (end: ">"))
line((0,2.4), (3,2.4), mark: (start: ">", end: ">"))
line((0,1.8), (3,1.8), mark: (symbol: ">")) // aux deux
bouts
line((0,1.2), (3,1.2), mark: (end: (">", ">"))) // marques
multiples
line((0,.6), (3,.6), mark: (end: ">", pos: 50%)) // au milieu
line((0,0), (3,0), mark: (end: ">", harpoon: true))
```



5.4 Taille, remplissage, inclinaison

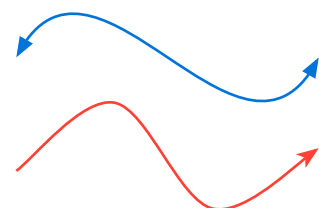
```
line((0,2.4), (3,2.4), mark: (end: ">", scale: 2))
line((0,1.8), (3,1.8), mark: (end: ">", length: .5, width: .5))
line((0,1.2), (3,1.2), mark: (end: ">", fill: red, stroke: red))
line((0,.6), (3,.6), mark: (end: ">", slant: 60%))
line((0,0), (3,0), mark: (end: ">", flip: true, harpoon: true))
```



Style de marque global `set-style(mark: (fill: black, scale: 1.4))` s'applique à toutes les marques suivantes. Les clés utiles : `length`, `width`, `inset`, `sep`, `scale`, `pos`, `offset`, `slant`, `harpoon`, `flip`, `reverse`, `anchor` ("tip", "center", "base"), `shorten-to`, `transform-shape`.

5.5 Marques sur les courbes

```
bezier((0,0), (4,0), (1,2), (3,-2),
  stroke: blue, mark: (start: ">", end: ">", fill: blue))
catmull((0,-1.5), (1.3,-.6), (2.6,-2), (4,-1.2),
  stroke: red, mark: (end: "stealth", fill: red))
```



5.6 mark comme élément autonome

```
line((0,0), (3,1), stroke: gray)
mark((0,0), (3,1), symbol: ">", scale: 3)
mark((1.5,.5), (3,1), symbol: "stealth", anchor: "center", scale:
3,
    fill: red, stroke: red)
```



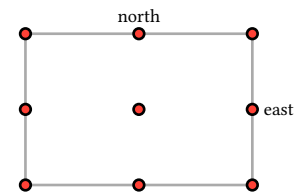
6 Ancres

Tout élément nommé (name: "x") expose des **ancres** que l'on cite "x.nom-de-l-ancre". Trois familles :

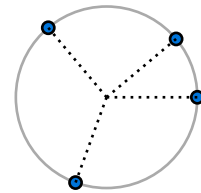
Famille	Valeurs
Boussole	north north-east east south-east south south-west west north-west center, ou un angle quelconque
Chemin	start mid end, ou un ratio / une longueur le long du chemin
Spécifiques	circle: center — arc: arc-start arc-end origin chord-center — line: sommets numérotés "0", "1", ... — content: boîte de texte

6.1 Ancres de boussole

```
rect((0,0), (3,2), name: "r", stroke: gray)
for a in ("north", "north-east", "east", "south-east", "south",
"south-west", "west", "north-west", "center") {
    circle("r." + a, radius: .07, fill: red)
}
content("r.north", anchor: "south", padding: .15, text(7pt)
[north])
content("r.east", anchor: "west", padding: .15, text(7pt)[east])
```

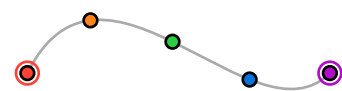


```
circle((0,0), radius: 1.2, name: "c", stroke: gray)
// une ancre peut être un angle arbitraire
for a in (0deg, 40deg, 130deg, 250deg) {
    circle("c." + repr(a), radius: .08, fill: blue)
    line("c.center", "c." + repr(a), stroke: (dash: "dotted"))
}
```



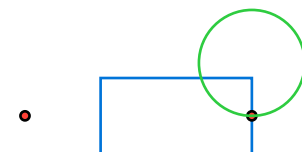
6.2 Ancres de chemin

```
bezier((0,0), (4,0), (1,2), (3,-1), name: "b", stroke: gray)
for (r, c) in ((0%, red), (25%, orange), (50%, green),
(75%, blue), (100%, purple)) {
    circle("b." + repr(r), radius: .09, fill: c)
}
circle("b.start", radius: .15, stroke: red)
circle("b.end", radius: .15, stroke: purple)
```



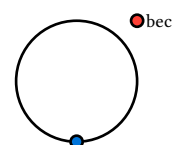
6.3 L'argument anchor: — placer l'élément par son ancre

```
circle((0,0), radius: .06, fill: red)
// l'ancre "north-west" du rect est posée sur (0,0)
rect((0,0), (2,1), anchor: "north-west", stroke: blue)
circle((3,0), radius: .06, fill: red)
circle((3,0), radius: .7, anchor: "south", stroke: green)
```



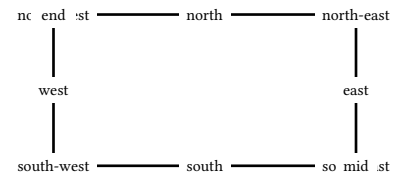
6.4 Créer ses propres ancres

```
group(name: "g", {
    circle((0,0), radius: .8)
    anchor("bec", (.8, .8))
    anchor("pied", (0, -.8))
})
circle("g.bec", radius: .08, fill: red)
circle("g.pied", radius: .08, fill: blue)
content("g.bec", anchor: "west", padding: .12, text(7pt)[bec])
```



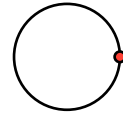
6.5 Parcourir les ancres : for-each-anchor

```
rect((0,0), (4,2), name: "r")
for-each-anchor("r", exclude: ("center", "default"), n => {
  circle((0,0), radius: .06, fill: red)
  content((0,0), text(6pt, n), anchor: "center",
    padding: .1, frame: "rect", fill: white, stroke: none)
})
```



6.6 copy-anchors

```
group(name: "outer", {
  circle((0,0), radius: .7, name: "inner")
  copy-anchors("inner") // outer hérite des ancres de inner
})
circle("outer.east", radius: .07, fill: red)
```

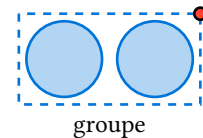


7 Groupes, calques et intersections

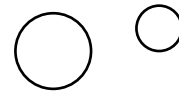
7.1 group

Un groupe isole le contexte (styles, transformations) et devient un élément nommable avec ses propres ancres.

```
group(name: "g", {
  set-style(fill: blue.lighten(70%), stroke: blue)
  circle((0,0), radius: .5)
  circle((1.2,0), radius: .5)
  rect((-0.6,-0.6), (1.8,.6), stroke: (dash: "dashed"), fill: none)
})
// le groupe a des ancres de boîte
circle("g.north-east", radius: .08, fill: red)
content("g.south", anchor: "north", padding: .15)[groupe]
```

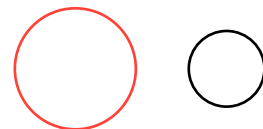


```
// padding et fond du groupe
group(padding: .3, fill: yellow.lighten(70%), stroke: orange, {
  circle((0,0), radius: .5)
  circle((1.4,.3), radius: .3)
})
```



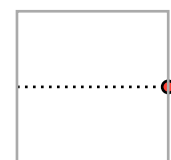
7.2 scope — isoler sans créer d'élément

```
scope({
  scale(1.6)
  stroke(red)
  circle((0,0), radius: .5)
})
circle((2,0), radius: .5) // ni mis à l'échelle, ni rouge
```



7.3 hide et floating

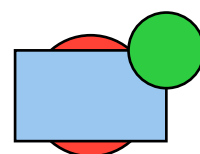
```
// hide : calculé (donc ancrable) mais non dessiné
hide(circle((0,0), radius: 1, name: "h"))
line((-1,0), (1,0), stroke: (dash: "dotted"))
circle((1,0), radius: .08, fill: red)
// floating : ne contribue pas aux bornes du canvas
floating(content((2.5,1.5))[hors bornes])
rect((-1,-1), (1,1), stroke: gray)
```



hors bornes

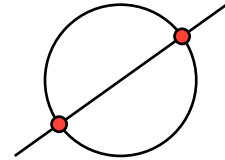
7.4 on-layer

```
// numéro de calque plus grand = dessiné au-dessus
rect((0,0), (2,1.2), fill: blue.lighten(60%))
on-layer(-1, circle((1,.6), radius: .8, fill: red)) // dessous
on-layer(1, circle((2,1.2), radius: .5, fill: green)) // dessus
```



7.5 intersections

```
intersections(name: "i", "x", {
  circle((0,0), radius: 1)
  line((-1.4,-1), (1.4,1))
})
for-each-anchor("x", n => {
  circle(), radius: .1, fill: red
})
```



Note `intersections("nom", { ... })` dessine son contenu et nomme chaque point d'intersection "nom.0", "nom.1", ... Utilisez `samples` : pour affiner la détection sur les courbes, et `sort` : pour ordonner les résultats.

7.6 get-ctx / set-ctx

```
get-ctx(ctx => {
  content((0,0), text(8pt)[unité = #ctx.length])
})
```

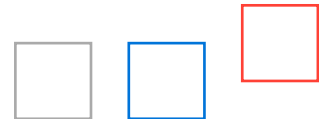
unité = 28.35pt

8 Transformations

Les transformations sont **cumulatives** et affectent tout ce qui suit dans le groupe courant. Enfermez-les dans un `scope` pour les limiter.

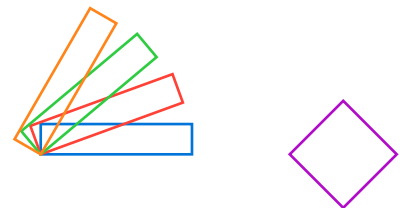
8.1 translate

```
rect((0,0), (1,1), stroke: gray)
scope({ translate((1.5, 0)); rect((0,0), (1,1), stroke: blue) })
scope({ translate(x: 3, y: .5); rect((0,0), (1,1), stroke: red) })
```



8.2 rotate

```
for (i, a) in (0deg, 20deg, 40deg, 60deg).enumerate() {
  scope({
    rotate(a)
    rect((0,0), (2,.4), stroke: (blue, red, green, orange).at(i))
  })
}
// rotation autour d'un point donné
scope({ rotate(45deg, origin: (4,0)); rect((3.5,-.5), (4.5,.5),
stroke: purple) })
```



8.3 scale

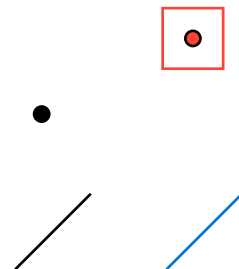
```
circle((0,0), radius: .5, stroke: gray)
scope({ scale(1.6); circle((0,0), radius: .5, stroke: blue) })
scope({ scale(x: 2, y: .5); circle((2,0), radius: .5, stroke:
red) })
```



8.4 set-origin et move-to

```
circle((0,0), radius: .1, fill: black)
set-origin((2,1)) // (0,0) est déplacé ici
circle((0,0), radius: .1, fill: red)
rect((- .4, - .4), (.4, .4), stroke: red)
```

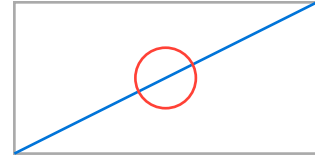
```
// move-to déplace seulement le "point précédent"
line((0,0), (1,1))
move-to((2,0))
line(), (rel: (1,1)), stroke: blue)
```



8.5 set-viewport

Remappe un rectangle sur un autre — pratique pour tracer dans des unités « métier ».

```
rect((0,0), (4,2), stroke: gray)
set-viewport((0,0), (4,2), bounds: (100, 50))
// désormais on dessine en coordonnées 0..100 × 0..50
line((0,0), (100,50), stroke: blue)
circle((50,25), radius: 10, stroke: red)
```



8.6 Matrices : set-transform / transform

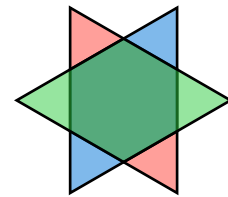
```
// cisaillement manuel
transform(((1, .4, 0, 0), (0, 1, 0, 0), (0, 0, 1, 0), (0, 0, 0, 1)))
rect((0,0), (2,1), stroke: blue)
```



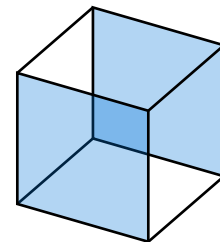
9 Dessin 3D

9.1 ortho — projection isométrique

```
ortho({
  on-xy({ rect((-1,-1), (1,1), fill: red.transparentize(50%)) })
  on-xz({ rect((-1,-1), (1,1), fill: green.transparentize(50%)) })
  on-zy({ rect((-1,-1), (1,1), fill: blue.transparentize(50%)) })
})
```

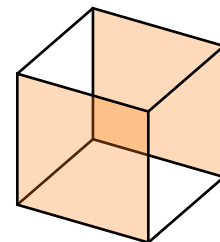


```
// un cube
ortho(x: 30deg, y: 30deg, {
  for z in (-1, 1) {
    on-xy(z: z, rect((-1,-1), (1,1), fill:
blue.transparentize(70%)))
  }
  for (x, y) in ((-1,-1), (1,-1), (1,1), (-1,1)) {
    line((x, y, -1), (x, y, 1))
  }
})
```



9.2 perspective

```
perspective(x: 30deg, y: 30deg, {
  for z in (-1, 1) {
    on-xy(z: z, rect((-1,-1), (1,1), fill:
orange.transparentize(70%)))
  }
  for (x, y) in ((-1,-1), (1,-1), (1,1), (-1,1)) {
    line((x, y, -1), (x, y, 1))
  }
})
```



Paramètres utiles ortho(x:, y:, z:, sorted: true, cull-face: none, flatten: false, ...) et perspective(x:, y:, z:, distance: auto, ...). cull-face: "cw" ou "ccw" supprime les faces arrière ; sorted trie les objets de l'arrière vers l'avant.

9.3 Plans de travail on-xy / on-xz / on-zy

```
ortho({
  on-xy(z: 0, { grid((-1,-1), (1,1), stroke: gray + .3pt)
    content((0,0))[xy] })
  on-xz(y: -1, { circle((0,0), radius: .6, stroke: red)
    content((0,0), text(fill: red)[xz]) })
})
```

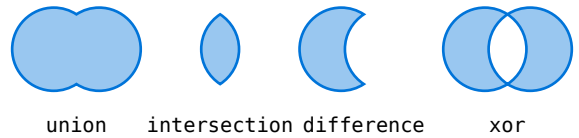


10 Opérations booléennes

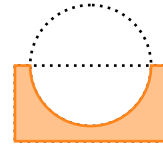
```
boolean(a, b, op: "difference", fill-rule-a: auto, fill-rule-b: auto, name: none, ..style)
```

Les opérandes sont soit des éléments, soit le **nom** d'un élément déjà dessiné. Tous les sous-chemins doivent être fermés et coplanaires.

```
let a = circle((0,0), radius: .55)
let b = circle((.6,0), radius: .55)
for (i, op) in ("union", "intersection", "difference",
"xor").enumerate() {
  scope({
    translate((i * 1.9, 0))
    boolean(a, b, op: op, fill: blue.lighten(60%), stroke:
blue)
    content((.3, -1), text(6pt, raw(op)))
  })
}
```



```
rect((-1,-1), (1,0), name: "r", stroke: (dash: "dotted"))
circle((0,0), radius: .8, name: "c", stroke: (dash: "dotted"))
boolean("r", "c", op: "difference", fill: orange.lighten(50%),
stroke: orange)
```

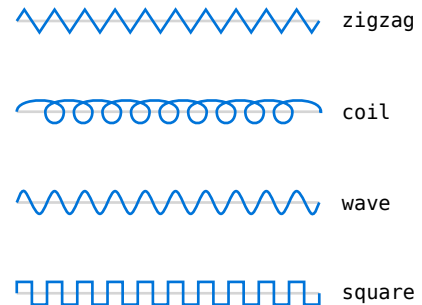


11 Décorations

Toutes les décorations prennent un **chemin cible** et acceptent amplitude, segments ou segment-length, start, stop, name.

11.1 zigzag, coil, wave, square

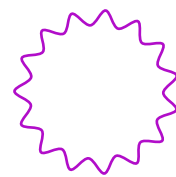
```
let fns = (decorations.zigzag, decorations.coil,
decorations.wave, decorations.square)
let names = ("zigzag", "coil", "wave", "square")
for (i, f) in fns.enumerate() {
  let y = -i * 1.2
  line((0, y), (4, y), stroke: gray.lighten(50%))
  f(line((0, y), (4, y)), amplitude: .3, stroke: blue)
  content((4.3, y), anchor: "west", text(7pt,
raw(names.at(i))))
}
```



```
// start / stop : portion décorée du chemin
line((0,0), (4,0), stroke: gray)
decorations.coil(line((0,0), (4,0)), amplitude: .3,
start: 20%, stop: 80%, segments: 8, stroke: red)
```

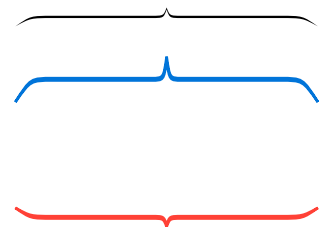


```
// une décoration marche sur n'importe quel chemin
decorations.wave(circle((0,0), radius: 1), amplitude: .2,
segment-length: .4, stroke: purple)
```

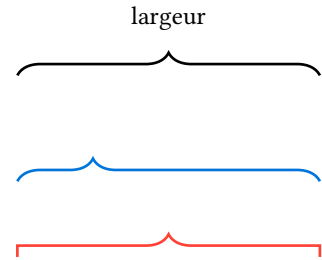


11.2 Accolades : brace et flat-brace

```
decorations.brace((0,0), (4,0))
decorations.brace((0,-1), (4,-1), amplitude: .6, stroke: blue)
decorations.brace((0,-2.4), (4,-2.4), flip: true, stroke: red)
```



```
decorations.flat-brace((0,0), (4,0), name: "b")
content("b.content", anchor: "south")[largeur]
decorations.flat-brace((0,-1.4), (4,-1.4), aspect: 25%, stroke:
blue)
decorations.flat-brace((0,-2.4), (4,-2.4), outer-curves: 0,
stroke: red)
```



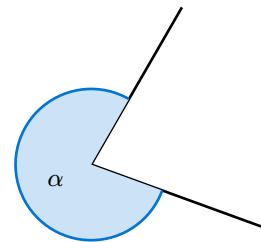
Ancres des accolades start, end, spike, content, center. L'ancre content est le point idéal pour poser l'étiquette, décalée devant la pointe.

12 Angles

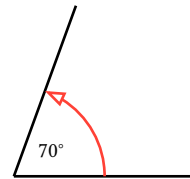
```
cetz.angle.angle(origin, a, b, direction: "ccw", label: none, name: none, ..style)
```

direction vaut "ccw" (défaut), "cw", "near" (angle intérieur) ou "far" (angle extérieur). Les ancres produites sont a, b, origin, label, start, end.

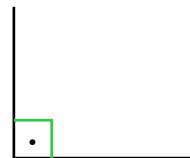
```
line((0,0), (60deg, 2.4), name: "a")
line((0,0), (-20deg, 2.4), name: "b")
angle-lib.angle("a.start", "a.end", "b.end", label: $alpha$,
fill: blue.lighten(80%), stroke: blue, radius: 1)
```



```
line((0,0), (70deg, 2.4), name: "a")
line((0,0), (0deg, 2.4), name: "b")
// étiquette calculée à partir de la valeur de l'angle
// ccw : de b vers a ; "near" prendrait toujours l'angle intérieur
angle-lib.angle("a.start", "b.end", "a.end", radius: 1.2,
mark: (end: ">"), stroke: red,
label: v => text(8pt)[#calc.round(v / 1deg)°])
```



```
line((0,0), (0,2), name: "a")
line((0,0), (2.4,0), name: "b")
angle-lib.right-angle("a.start", "a.end", "b.end", radius: .5,
stroke: green)
```

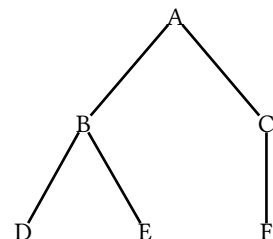


13 Arbres

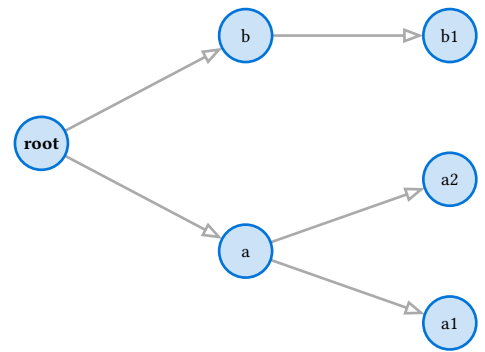
```
cetz.tree.tree(root, draw-node: auto, draw-edge: auto, direction: "down", grow: 1, spread: 1, name: none)
```

Un arbre est un tableau imbriqué : (racine, enfant1, enfant2, ...).

```
tree.tree(([A], ([B], [D], [E]), ([C], [F])), spread: 1.4, grow:
1.2)
```



```
tree.tree(
  ([*root*], ([a], [a1], [a2]), ([b], [b1])),
  direction: "right", grow: 2, spread: 1.2,
  // node : dictionnaire avec .content, .depth, .group-name
  draw-node: node => {
    circle((0,0), radius: .35, fill: blue.lighten(80%), stroke:
blue)
    content((0,0), text(7pt, node.content))
  },
  // parent / child : nœuds ; .group-name donne le nom de
l'élément
  draw-edge: (parent, child) => {
    line(parent.group-name, child.group-name,
      mark: (end: ">"), stroke: gray)
  })
```

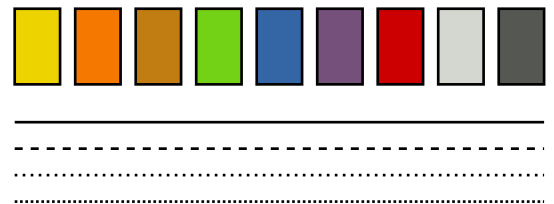


14 Palettes

```
let p = palette.new(colors: (red, blue, green, orange, purple))
for i in range(0, p("len")) {
  set-style(..p(i))
  circle((i * 1.1, 0), radius: .45)
}
```

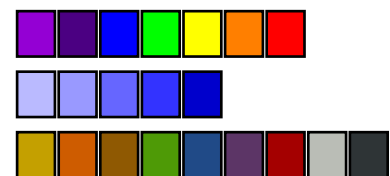


```
// palettes prédéfinies + variation par trait
let p = palette.new(colors: palette.tango-colors)
for i in range(0, p("len")) {
  rect((i * .8, 0), (i * .8 + .6, 1), ..p(i))
}
let d = palette.new(colors: (black,),
  dash: ("solid", "dashed", "dotted",
"densely-dotted"))
for i in range(0, d("len")) {
  line((0, -.5 - i * .35), (7, -.5 - i * .35), ..d(i, fill:
false, stroke: true))
}
```



Palettes prêtes à l'emploi cetz.palette expose directement gray, red, orange, light-green, dark-green, turquoise, cyan, blue, indigo, purple, magenta, pink, rainbow, tango, tango-light, tango-dark, ainsi que les listes de couleurs correspondantes (tango-colors,...). Utilisez `p(i, fill: false, stroke: true)` pour colorer le trait plutôt que le fond, et `p("len")` pour connaître le nombre de variantes.

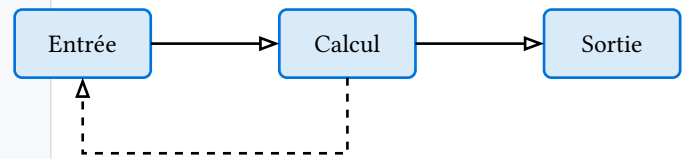
```
for (i, p) in (palette.rainbow, palette.blue, palette.tango-
dark).enumerate() {
  for j in range(0, p("len")) {
    rect((j * .55, -i * .8), (j * .55 + .5, -i * .8
+ .6), ..p(j))
  }
}
```



15 Recettes

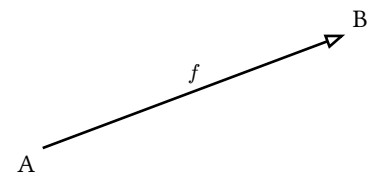
15.1 Diagramme de boîtes et flèches

```
let box(pos, name, body) = {
  rect((rel: (-.9, -.45), to: pos), (rel: (.9, .45), to: pos),
    name: name, radius: .1, fill: blue.lighten(85%),
    stroke: blue)
  content(name, body)
}
box((0,0), "a", [Entrée])
box((3.5,0), "b", [Calcul])
box((7,0), "c", [Sortie])
line("a.east", "b.west", mark: (end: ">"))
line("b.east", "c.west", mark: (end: ">"))
line("b.south", (rel: (0,-1)), (rel: (-3.5,0)), "a.south",
  mark: (end: ">"), stroke: (dash: "dashed"))
```



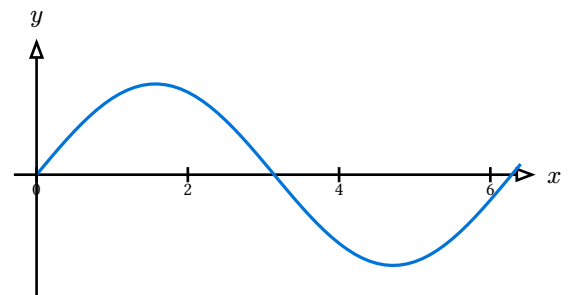
15.2 Étiqueter une flèche

```
line((0,0), (4,1.5), name: "l", mark: (end: ">"))
content("l.50%", anchor: "south", padding: .1, text(8pt)[$f$])
content("l.start", anchor: "north-east", padding: .1)[A]
content("l.end", anchor: "south-west", padding: .1)[B]
```



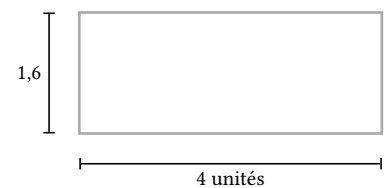
15.3 Repère et courbe de fonction

```
let f = x => calc.sin(x) * 1.2
line((-3,0), (6.6,0), mark: (end: ">"))
line((0,-1.6), (0,1.8), mark: (end: ">"))
content((6.6,0), anchor: "west", padding: .15, $x$)
content((0,1.8), anchor: "south", padding: .15, $y$)
let pts = range(0, 65).map(i => {
  let x = i / 10
  (x, f(x))
})
line(.pts, stroke: blue + 1.2pt)
for x in (0, 2, 4, 6) {
  line((x, -.1), (x, .1))
  content((x, -.1), anchor: "north", text(7pt)[#x])
}
```



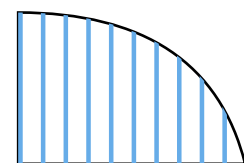
15.4 Cotation

```
rect((0,0), (4,1.6), stroke: gray)
set-style(mark: (start: "|", end: "|"), stroke: .6pt)
line((0,-.4), (4,-.4), name: "w")
content("w.50%", anchor: "north", padding: .1, text(8pt)[4
unités])
line((-4,0), (-4,1.6), name: "h")
content("h.50%", anchor: "east", padding: .1, text(8pt)[1,6])
```



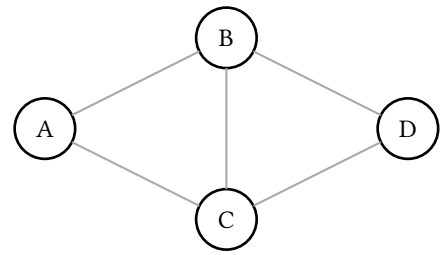
15.5 Hachures d'une zone (via boolean)

```
let region = merge-path(close: true, {
  line((0,0), (3,0))
  bezier((3,0), (0,2), (2.5,2))
})
let hatch = compound-path({
  for i in range(-3, 14) {
    let x = i * .3
    rect((x, 0), (x + .06, 2.2))
  }
})
region
boolean(region, hatch, op: "intersection", fill:
blue.lighten(40%),
stroke: none)
```



15.6 Graphe avec nœuds circulaires

```
let nodes = (("A", (0,0)), ("B", (2.4,1.2)), ("C",
(2.4,-1.2)), ("D", (4.8,0)))
let edges = (("A","B"), ("A","C"), ("B","D"), ("C","D"),
("B","C"))
for (n, p) in nodes {
  circle(p, radius: .4, name: n, fill: white, stroke: 1pt)
  content(p, n)
}
for (a, b) in edges { line(a, b, stroke: gray + .8pt) }
```



Astuce Une ligne entre deux éléments **nommés** est automatiquement raccourcie jusqu'à leur bordure : `line("A", "B")` ne traverse pas les cercles.

15.7 Placer un dessin dans le texte

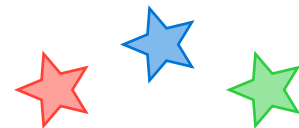
```
Un symbole en ligne #cetz.canvas(length: .35cm, baseline: (0, .2),
{
  import cetz.draw: *
  circle((0,0), radius: 1, fill: red.lighten(60%), stroke: red)
}) au milieu d'un paragraphe.
```

Un symbole en ligne  au milieu d'un paragraphe.

baseline `canvas(baseline: <coordonnée>)` aligne la ligne de base du dessin sur cette coordonnée du repère, ce qui permet de l'insérer proprement dans une ligne de texte. Donnez un couple `(0, .2)` : en 0.5.2, la forme scalaire déclenche une erreur « variables from outside the context expression are read-only ».

15.8 Réutiliser un motif

```
// une fonction qui renvoie des éléments = un "motif" paramétrable
let star-at(pos, c) = n-star(pos, 5, radius: .5, inner-radius:
45%,
                                fill: c.lighten(50%), stroke: c)
star-at((0,0), red)
star-at((1.4,.6), blue)
star-at((2.8,0), green)
```



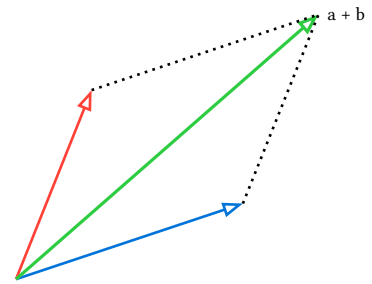
16 Vecteurs et matrices

CeTZ expose ses utilitaires mathématiques : indispensables dès que l'on calcule des positions plutôt que de les écrire à la main.

16.1 cetz.vector

Fonction	Effet
<code>add(a, b)</code> <code>sub(a, b)</code>	somme / différence
<code>scale(v, x)</code> <code>div(v, x)</code>	produit / division par un scalaire
<code>neg(v)</code>	opposé
<code>len(v)</code> <code>dist(a, b)</code>	norme / distance
<code>norm(v)</code>	vecteur unitaire
<code>dot(a, b)</code> <code>cross(a, b)</code>	produit scalaire / vectoriel
<code>angle2(a, b)</code>	angle du vecteur $b - a$ (2D)
<code>angle(a, c, b)</code>	angle en c entre a et b
<code>lerp(a, b, t)</code>	interpolation linéaire
<code>element-product(a, b)</code>	produit terme à terme
<code>as-vec(v)</code>	complète en vecteur 3D

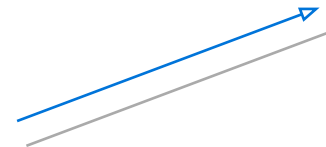
```
let a = (3, 1, 0)
let b = (1, 2.5, 0)
line((0,0), a, mark: (end: ">"), stroke: blue)
line((0,0), b, mark: (end: ">"), stroke: red)
// somme : règle du parallélogramme
line((0,0), vector.add(a, b), mark: (end: ">"), stroke: green + 1.2pt)
line(a, vector.add(a, b), stroke: (dash: "dotted"))
line(b, vector.add(a, b), stroke: (dash: "dotted"))
content(vector.add(a, b), anchor: "west", padding: .12,
text(7pt)[a + b])
```



```
let a = (3, 1)
// normale unitaire : tourner de 90° puis normaliser
let n = vector.norm((-a.at(1), a.at(0), 0))
line((0,0), a, mark: (end: ">"))
line((0,0), n, mark: (end: ">"), stroke: red)
content(n, anchor: "east", padding: .1, text(7pt)[normale])
get-ctx(_ => none)
content((3.4, 1), anchor: "west",
text(7pt)[|a| = #calc.round(vector.len(a), digits: 3)])
```



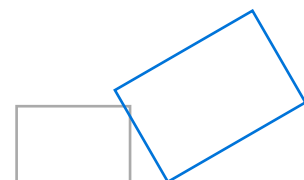
```
// une flèche décalée parallèlement à un segment
let (a, b) = ((0,0), (4,1.5))
let d = vector.norm(vector.sub(b, a))
let n = vector.scale((-d.at(1), d.at(0), 0), .35)
line(a, b, stroke: gray)
line(vector.add(a, n), vector.add(b, n),
mark: (end: ">"), stroke: blue)
```



16.2 cetz.matrix

Fonction	Effet
ident(4) diag(..)	identité / diagonale
mul-mat(a, b, ..)	produit matriciel
mul-vec(m, v)	applique la matrice à un vecteur
inverse(m)	inverse
transform-translate(x, y, z)	translation
transform-scale(f)	homothétie
transform-rotate-x/y/z(a)	rotation autour d'un axe
transform-rotate-xyz(x, y, z)	rotation composée
transform-rotate-aer(az, el, roll)	azimut / élévation / roulis
transform-shear-x(f) -z(f)	cisaillement

```
// composer soi-même une transformation
let m = matrix.mul-mat(
  matrix.transform-translate(2, 0, 0),
  matrix.transform-rotate-z(30deg),
  matrix.transform-scale(1.4),
)
rect((0,0), (1.5,1), stroke: gray)
transform(m)
rect((0,0), (1.5,1), stroke: blue + 1pt)
```



Ordre des produits mul-mat(A, B) applique **B d'abord**, puis A — comme en algèbre linéaire. Un transform(m) s'ajoute à la transformation courante ; set-transform(m) la remplace.

17 Étendre CeTZ

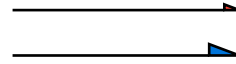
17.1 Marques personnalisées : register-mark

register-mark(symbol, body, mnemonic: none, tip: none, base: none, ..)

body est une fonction style => éléments. Les ancres tip et base disent à CeTZ où la pointe se situe et où le chemin doit s'arrêter.

```
register-mark("flag", style => {
  anchor("tip", (0, 0))
  anchor("base", (style.length, 0))
  line((0,0), (style.length, 0),
    (style.length, style.width / 2), close: true,
    fill: style.fill, stroke: style.stroke)
}, mnemonic: "F")

line((0,1), (3,1), mark: (end: "flag", fill: red))
line((0,.4), (3,.4), mark: (end: "F", fill: blue, scale: 2))
```



Piège vérifié en 0.5.2 Déclarez les ancres tip/base à l'intérieur de body, avec anchor(.). Les paramètres homonymes tip: et base: de register-mark construisent bien les ancres, mais la fonction insère body au lieu de la version augmentée : ils restent sans effet et la marque échoue avec dictionary does not contain key "tip".

17.2 Systèmes de coordonnées personnalisés

register-coordinate-resolver(resolver)

Le résolveur est une fonction (ctx, coord) => coord. Il est **scopé** au groupe courant.

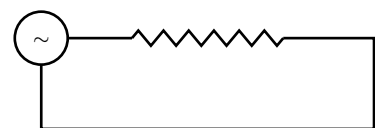
```
// un système de coordonnées logarithmique
register-coordinate-resolver((ctx, c) => {
  if type(c) == dictionary and "log" in c {
    c = c.log.map(n => calc.log(n, base: 10))
  }
  return c
})
line((0,0), (4,0), mark: (end: ">"))
for v in (1, 10, 100, 1000, 10000) {
  circle((log: (v, 1)), radius: .1, fill: blue)
  content((log: (v, 1)), anchor: "north", padding: .2,
  text(6pt)[#v])
}
```



17.3 Éléments réutilisables

Un « élément » CeTZ n'est rien d'autre qu'une fonction qui renvoie des éléments. Enveloppez-la dans un group nommé pour lui donner des ancres.

```
let resistor(a, b, name: none) = group(name: name, {
  decorations.zigzag(line(a, b), amplitude: .2, segments: 6)
  anchor("west", a)
  anchor("east", b)
})
let source(pos, name: none) = group(name: name, {
  circle(pos, radius: .35)
  content(pos, text(8pt)[$~$])
})
source((0,0), name: "s")
resistor((1.2,0), (3.2,0), name: "r1")
line("s.east", "r1.west")
line("r1.east", (4.4,0), (4.4,-1.2), (0,-1.2), "s.south")
```



17.4 Interroger le contexte

```
get-ctx(ctx => {
  // ctx.length, ctx.style, ctx.transform, ctx.nodes ...
  content((0,.4), text(7pt)[unité : #ctx.length])
  content((0,0), text(7pt)[éléments connus :
  #ctx.nodes.keys().len()])
})
circle((0,0), radius: .01, name: "invisible")
```

unité : 28.35pt
éléments connus : 0

set-ctx set-ctx(ctx => { ctx.foo = 1; ctx }) permet de stocker ses propres données dans le contexte, lisibles plus loin via get-ctx. Utile pour écrire des bibliothèques d'éléments qui se coordonnent entre eux.

18 L'écosystème : cetz-plot

cezt-plot 0.1.4 est le module de tracé officiel, compatible CeTZ 0.5.x.

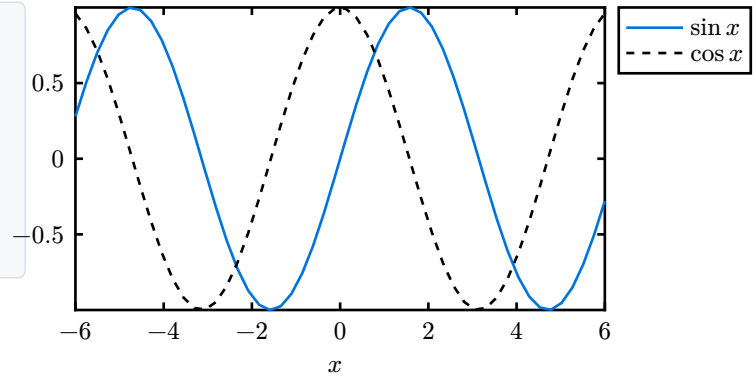
```
#import "@preview/cetz-plot:0.1.4": plot, chart, smartart
```

Dans les exemples ci-dessous, plot, chart et smartart désignent ces modules, et le tout vit dans un `cezt.canvas`.

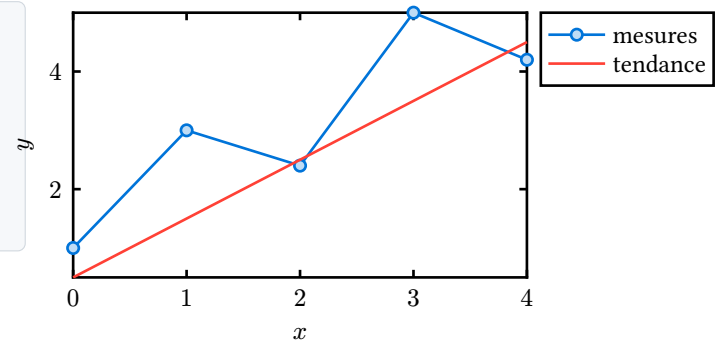
18.1 Courbes de fonctions

```
plot.plot(body, size: (1,1), axis-style: "scientific", x-tick-step: auto, ..)
```

```
// #import "@preview/cetz-plot:0.1.4": plot
plot.plot(size: (7, 4), x-tick-step: 2, y-tick-step: .5,
  x-label: $x$, y-label: $y$, {
  plot.add(domain: (-6, 6), x => calc.sin(x), label:
    $sin x$)
  plot.add(domain: (-6, 6), x => calc.cos(x), label:
    $cos x$,
    style: (stroke: (dash: "dashed"))))
})
```

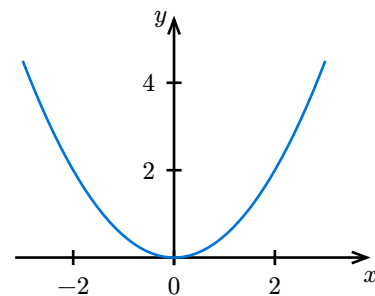


```
// #import "@preview/cetz-plot:0.1.4": plot
// données brutes + marques
plot.plot(size: (6, 3.5), x-tick-step: 1, y-tick-step: 2, {
  plot.add(((0,1), (1,3), (2,2.4), (3,5), (4,4.2)),
    mark: "o", mark-size: .15, label: "mesures")
  plot.add(((0,0.5), (4,4.5)), style: (stroke: red),
    label: "tendance")
})
```



18.2 Styles d'axes

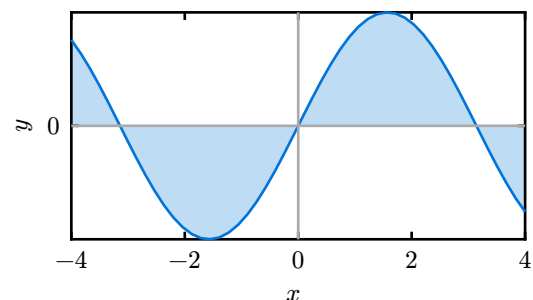
```
// #import "@preview/cetz-plot:0.1.4": plot
plot.plot(size: (4, 2.6), axis-style: "school-book",
  x-tick-step: 2, y-tick-step: 2, {
  plot.add(domain: (-3, 3), x => x * x / 2)
})
```



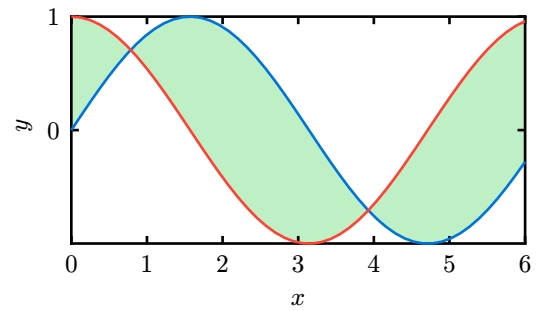
Valeurs possibles : "scientific" (cadre complet, défaut), "school-book" (axes en croix fléchés), "left", "right", none.

18.3 Remplissages et zones

```
// #import "@preview/cetz-plot:0.1.4": plot
plot.plot(size: (6, 3), x-tick-step: 2, y-tick-step: 1,
  {
  plot.add(domain: (-4, 4), x => calc.sin(x), fill:
    true,
    style: (fill: blue.transparentize(75%),
    stroke: blue))
  plot.add-hline(0, style: (stroke: gray))
  plot.add-vline(0, style: (stroke: gray))
})
```

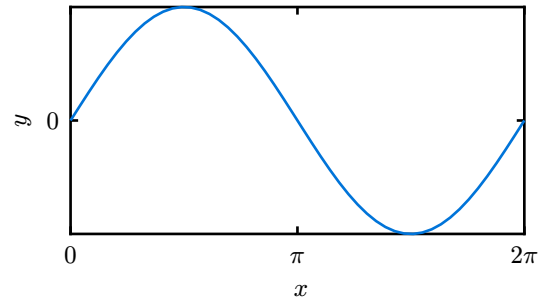


```
// #import "@preview/cetz-plot:0.1.4": plot
plot.plot(size: (6, 3), x-tick-step: 1, y-tick-step: 1,
{
  let f = x => calc.sin(x)
  let g = x => calc.cos(x)
  // les deux opérandes sont des données ou des
  fonctions, pas des plot.add
  plot.add-fill-between(f, g, domain: (0, 6),
    style: (fill: green.transparentize(70%), stroke:
none))
  plot.add(domain: (0, 6), f, style: (stroke: blue))
  plot.add(domain: (0, 6), g, style: (stroke: red))
})
```



18.4 Échelles et formats de graduation

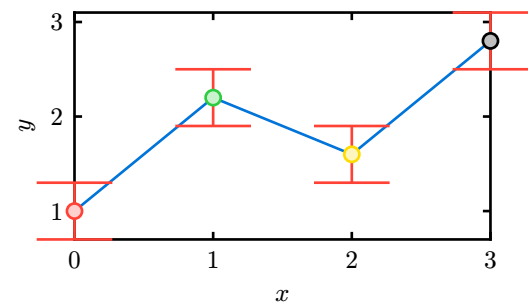
```
// #import "@preview/cetz-plot:0.1.4": plot
plot.plot(size: (6, 3), x-tick-step: calc.pi, y-tick-
step: 1,
  x-format: plot.formats.multiple-of, {
    plot.add(domain: (0, 2 * calc.pi), calc.sin)
  })
```



Les formateurs disponibles : `plot.formats.decimal`, `sci` (notation scientifique), `fraction`, `multiple-of` (multiples de π par défaut).

18.5 Barres d'erreur, aires, contours

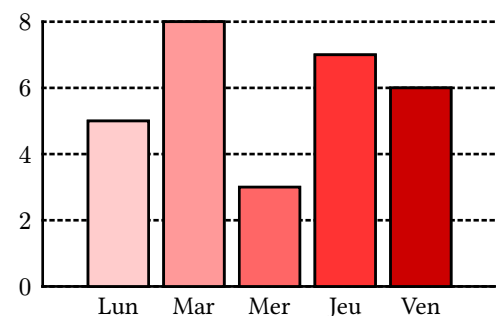
```
// #import "@preview/cetz-plot:0.1.4": plot
plot.plot(size: (5.5, 3), x-tick-step: 1, y-tick-step:
1, {
  let d = ((0,1), (1,2.2), (2,1.6), (3,2.8))
  plot.add(d, mark: "o", mark-size: .12)
  for (x, y) in d {
    plot.add-errorbar((x, y), y-error: .3, style:
(stroke: red))
  }
})
```



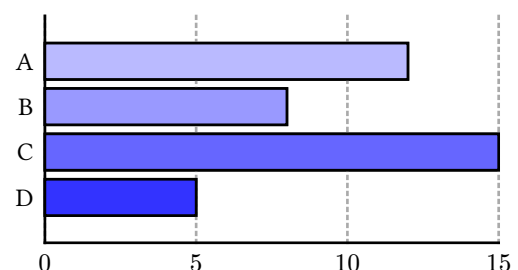
19 Diagrammes : chart

19.1 Barres et colonnes

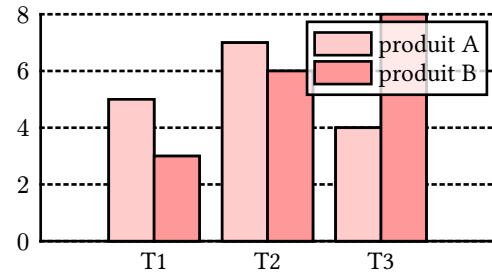
```
// #import "@preview/cetz-plot:0.1.4": chart
let data = (("Lun", 5), ("Mar", 8), ("Mer", 3), ("Jeu",
7), ("Ven", 6))
chart.columnchart(data, size: (6, 3.5), y-tick-step: 2)
```



```
// #import "@preview/cetz-plot:0.1.4": chart
let data = (("A", 12), ("B", 8), ("C", 15), ("D", 5))
chart.barchart(data, size: (6, 3), x-tick-step: 5,
  bar-style: cetz.palette.blue)
```

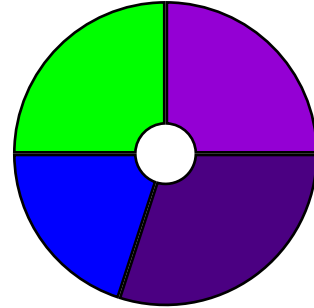


```
// #import "@preview/cetz-plot:0.1.4": chart
// barres groupées : mode "clustered"
let data = (("T1", 5, 3), ("T2", 7, 6), ("T3", 4, 8))
chart.columnchart(data, mode: "clustered", value-key:
(1, 2),
size: (6, 3), y-tick-step: 2,
labels: ([produit A], [produit B]),
legend: "inner-north-east")
```



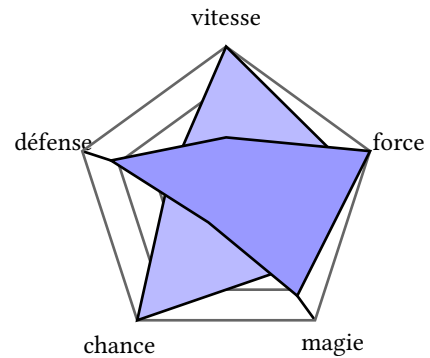
19.2 Camembert

```
// #import "@preview/cetz-plot:0.1.4": chart
chart.piechart((25, 30, 20, 25), radius: 2,
slice-style: cetz.palette.rainbow,
inner-radius: .4,
label-key: none)
```



19.3 Radar

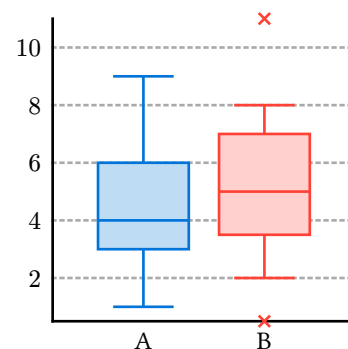
```
// #import "@preview/cetz-plot:0.1.4": chart
// radarchart(labels, data) : étiquettes en premier
chart.radarchart(
([vitesse], [force], [magie], [chance], [défense]),
((5, 4, 3, 5, 2).map(v => v / 5),
(2, 5, 4, 1, 4).map(v => v / 5)),
data-style: cetz.palette.blue)
```



Normalisez entre 0 et 1 Malgré une assertion qui autorise $0 \leq \text{valeur} \leq \text{radius}$, les valeurs sont interprétées comme une **fraction du rayon** en 0.1.4 : au-delà de 1, la toile de fond et les étiquettes disparaissent et le tracé déborde. Divisez donc toujours vos données par leur maximum. Styles utiles : `web-style`, `web-ticks`, `web-label-offset`, `radius`, `center-pos`.

19.4 Boîte à moustaches

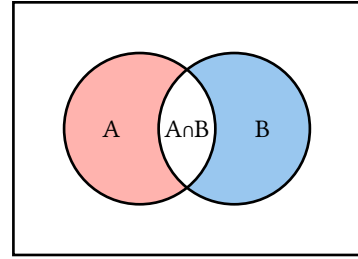
```
// #import "@preview/cetz-plot:0.1.4": chart
chart.boxwhisker((
(label: [A], min: 1, q1: 3, q2: 4, q3: 6, max: 9),
(label: [B], min: 2, q1: 3.5, q2: 5, q3: 7, max: 8,
outliers: (11, .5)),
), size: (4, 4), label-key: "label", y-tick-step: 2)
```



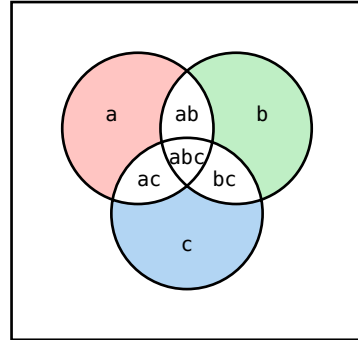
20 Diagrammes de Venn : cetz-venn

```
#import "@preview/cetz-venn:0.2.0" as venn
```

```
// #import "@preview/cetz-venn:0.2.0" as venn
venn.venn2(name: "v", a-fill: red.transparentize(60%),
           b-fill: blue.transparentize(60%))
content("v.a", [A])
content("v.b", [B])
content("v.ab", [A∩B])
```



```
// #import "@preview/cetz-venn:0.2.0" as venn
venn.venn3(name: "v",
           a-fill: red.transparentize(70%),
           b-fill: green.transparentize(70%),
           c-fill: blue.transparentize(70%))
for k in ("a", "b", "c", "ab", "bc", "ac", "abc") {
  content("v." + k, text(7pt, raw(k)))
}
```

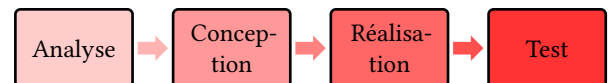


Attributs de set Chaque ensemble accepte `*-fill`, `*-stroke`, `*-layer`, `*-radius`, `*-distance`, `*-anchor-outset`, préfixés par son nom : `a-fill`, `b-radius`, ... Les noms d'ensembles sont aussi des **ancres** : `a`, `b`, `ab`, `not-ab` (et `c`, `bc`, `ac`, `abc`, `not-abc` pour `venn3`).

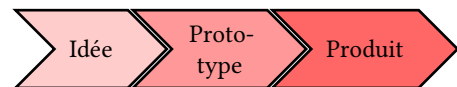
21 SmartArt : processus et cycles

21.1 Processus linéaire

```
// #import "@preview/cetz-plot:0.1.4": smartart
smartart.process.basic(
  ([Analyse], [Conception], [Réalisation], [Test]),
  equal-width: true, equal-height: true)
```

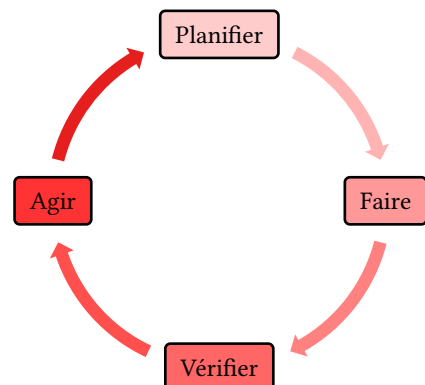


```
// #import "@preview/cetz-plot:0.1.4": smartart
smartart.process.chevron(
  ([Idée], [Prototype], [Produit]),
  equal-width: true)
```



21.2 Cycle

```
// #import "@preview/cetz-plot:0.1.4": smartart
smartart.cycle.basic(
  ([Planifier], [Faire], [Vérifier], [Agir]),
  radius: 2.2)
```



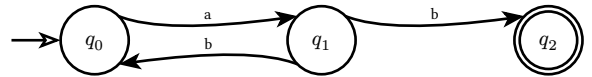
22 Recettes avancées

22.1 Automate à états finis

```
let state(pos, name, label, accept: false) = {
  circle(pos, radius: .45, name: name, fill: white)
  if accept { circle(pos, radius: .38) }
  content(pos, text(8pt, label))
}

let trans(a, b, label, bend: 25deg) = {
  let m = (a, 50%, b)
  bezier(a, b, (m, .6, bend, a), mark: (end: "stealth",
fill: black),
  name: "t")
  content("t.50%", anchor: "south", padding: .08, text(6pt,
label))
}

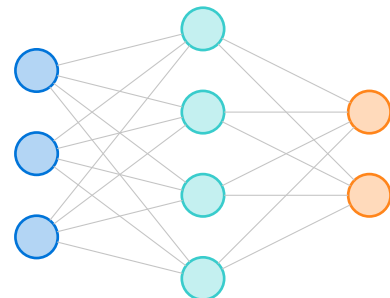
state((0,0), "q0", $q_0$)
state((3,0), "q1", $q_1$)
state((6,0), "q2", $q_2$, accept: true)
line((-1.1,0), "q0.west", mark: (end: "stealth"))
trans("q0.north-east", "q1.north-west", "a")
trans("q1.north-east", "q2.north-west", "b")
trans("q1.south-west", "q0.south-east", "b")
```



22.2 Réseau de neurones

```
let layers = (3, 4, 2)
for (l, n) in layers.enumerate() {
  for i in range(n) {
    let y = (n - 1) / 2 - i
    circle((l * 2.2, y * 1.1), radius: .28,
    fill: (blue, teal, orange).at(l).lighten(70%),
    stroke: (blue, teal, orange).at(l),
    name: "n" + str(l) + "-" + str(i))
  }
}

for l in range(layers.len() - 1) {
  for i in range(layers.at(l)) {
    for j in range(layers.at(l + 1)) {
      line("n" + str(l) + "-" + str(i),
      "n" + str(l + 1) + "-" + str(j),
      stroke: gray.lighten(30%) + .4pt)
    }
  }
}
```

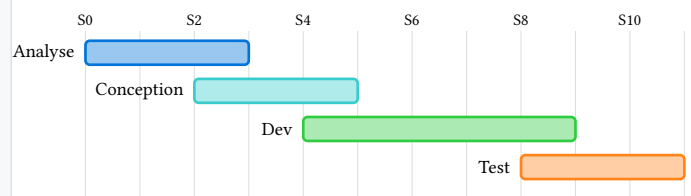


22.3 Diagramme de Gantt

```
let tasks = (
  ("Analyse", 0, 3, blue),
  ("Conception", 2, 5, teal),
  ("Dev", 4, 9, green),
  ("Test", 8, 11, orange),
)

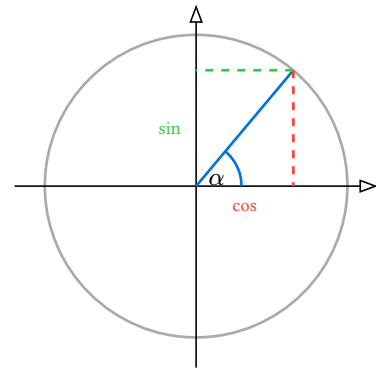
// grille et échelle
for x in range(0, 12) {
  line((x, .4), (x, -tasks.len() * .7), stroke:
gray.lighten(60%) + .3pt)
  if calc.rem(x, 2) == 0 {
    content((x, .5), text(6pt)[S#x], anchor: "south")
  }
}

for (i, (nom, a, b, c)) in tasks.enumerate() {
  let y = -i * .7
  rect((a, y - .22), (b, y + .22), radius: .08,
  fill: c.lighten(60%), stroke: c)
  content((a - .2, y), anchor: "east", text(7pt, nom))
}
```



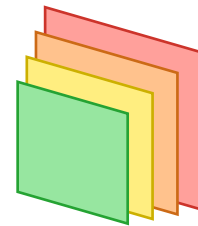
22.4 Cercle trigonométrique

```
let a = 50deg
circle((0,0), radius: 2, stroke: gray)
line((-2.4,0), (2.4,0), mark: (end: ">"), stroke: .6pt)
line((0,-2.4), (0,2.4), mark: (end: ">"), stroke: .6pt)
line((0,0), (a, 2), stroke: blue + 1pt, name: "r")
// projections
line("r.end", (calc.cos(a) * 2, 0),
  stroke: (paint: red, dash: "dashed"))
line("r.end", (0, calc.sin(a) * 2),
  stroke: (paint: green, dash: "dashed"))
content((calc.cos(a), -.25), text(7pt, fill: red)[cos])
content((-0.35, calc.sin(a)), text(7pt, fill: green)[sin])
angle-lib.angle((0,0), (1,0), "r.end", radius: .6,
  label: $alpha$, stroke: blue)
```



22.5 Empilement 3D avec tri de profondeur

```
ortho(x: 30deg, y: 30deg, {
  for (i, c) in (red, orange, yellow, green).enumerate() {
    let z = i * .45
    on-xy(z: z, {
      rect((-1.2 + i * .12, -1.2 + i * .12),
        (1.2 - i * .12, 1.2 - i * .12),
        fill: c.lighten(50%), stroke: c.darken(20%))
    })
  }
})
```



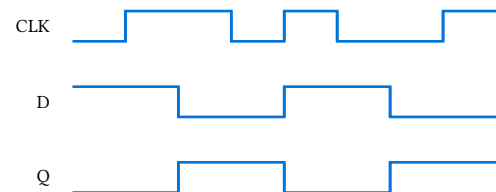
22.6 Carte de chaleur

```
let data = ((1, 5, 3, 8), (7, 2, 9, 4), (3, 6, 1, 7))
let mx = 9
for (i, row) in data.enumerate() {
  for (j, v) in row.enumerate() {
    rect((j * .8, -i * .8), (j * .8 + .8, -i * .8 - .8),
      fill: blue.transparentize(100% - v / mx * 100%),
      stroke: white + 1pt)
    content((j * .8 + .4, -i * .8 - .4),
      text(7pt, fill: if v > 5 { white } else
        { black })[#v])
  }
}
```

1	5	3	8
7	2	9	4
3	6	1	7

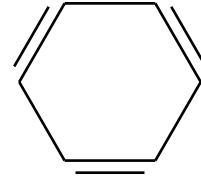
22.7 Chronogramme (signaux logiques)

```
let signal(y, bits, label) = {
  content((-0.3, y + .2), anchor: "east", text(7pt, label))
  let pts = ()
  for (i, b) in bits.enumerate() {
    pts.push((i * .7, y + b * .4))
    pts.push((i * .7 + .7, y + b * .4))
  }
  line(..pts, stroke: blue + 1pt)
  for i in range(1, bits.len()) {
    if bits.at(i) != bits.at(i - 1) {
      line((i * .7, y), (i * .7, y + .4), stroke: blue +
        1pt)
    }
  }
}
signal(0, (0,1,1,0,1,0,0,1), "CLK")
signal(-1, (1,1,0,0,1,1,0,0), "D")
signal(-2, (0,0,1,1,0,0,1,1), "Q")
```



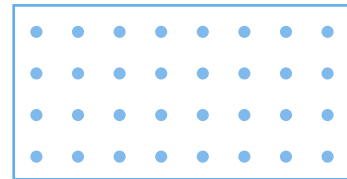
22.8 Molécule / graphe chimique

```
// la 2e liaison est décalée vers l'intérieur du cycle :
// on raccourcit aussi ses extrémités pour l'esthétique
let bond(a, b, double: false) = {
  line(a, b, stroke: 1pt)
  if double {
    let d = vector.norm(vector.sub(b, a))
    let n = vector.scale((-d.at(1), d.at(0), 0), -.16)
    line(vector.add(vector.lerp(a, b, .12), n),
          vector.add(vector.lerp(a, b, .88), n), stroke:
1pt)
  }
}
// hexagone du benzène – vector.* exige des points
cartésiens,
// on convertit donc les coordonnées polaires nous-mêmes
let r = 1.2
let pts = range(6).map(i => {
  let a = 60deg * i
  (calc.cos(a) * r, calc.sin(a) * r, 0)
})
for i in range(6) {
  bond(pts.at(i), pts.at(calc.rem(i + 1, 6)),
        double: calc.rem(i, 2) == 0)
}
```



22.9 Papier millimétré / motif répété

```
// motif de fond réutilisable via un groupe et des
transformations
let tile = {
  circle((0,0), radius: .08, fill: blue.lighten(50%),
stroke: none)
}
for i in range(0, 8) {
  for j in range(0, 4) {
    scope({
      translate((i * .55, j * .55))
      tile
    })
  }
}
rect((-0.3,-0.3), (4.15,2.0), stroke: blue.lighten(40%))
```

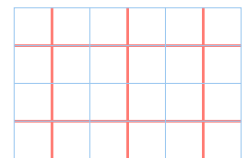
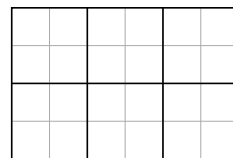


23 Showcase : détails souvent ignorés

Cette section rassemble les options fines que l'on découvre tard, une fois qu'on a lu le code source. Chaque exemple isole **une** subtilité.

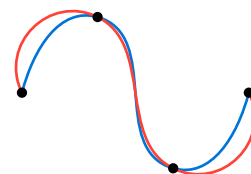
23.1 grid : shift, help-lines, pas asymétrique

```
grid((0,0), (3,2), step: .5, help-lines: true)
grid((0,0), (3,2), step: 1, stroke: black + .6pt)
// shift décale les lignes sans bouger les bornes
grid((4,0), (7,2), step: 1, shift: .5,
stroke: red.lighten(30%))
grid((4,0), (7,2), step: (x: 1, y: .5),
stroke: blue.lighten(60%) + .3pt)
```

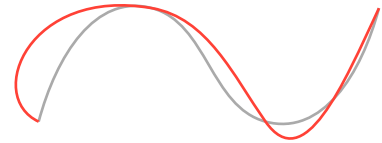


23.2 hobby : omega et tensions par segment

```
let p = ((0,0), (1,1), (2,-1), (3,0))
hobby(..p, omega: 0, stroke: blue) // proche de la droite
hobby(..p, omega: 1, stroke: red) // proche de l'arc de
cerle
for q in p { circle(q, radius: .05, fill: black) }
```

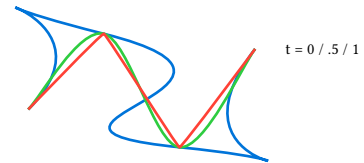


```
let p = ((0,0), (1.5,1.5), (3,0), (4.5,1.5))
// tension entrante / sortante, une valeur par segment
hobby(..p, ta: (1, 1, 1), tb: (1, 1, 1), stroke: gray)
hobby(..p, ta: (2, .5, 2), tb: (.5, 2, .5), stroke: red + 1pt)
```

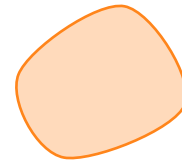


23.3 catmull : tension et fermeture

```
let p = ((0,0), (1,1), (2,-.5), (3,.8))
for (t, c) in ((0, blue), (.5, green), (1, red)) {
  catmull(..p, tension: t, stroke: c)
}
content((3.4, .8), anchor: "west", text(6pt)[t = 0 / .5 / 1])
```

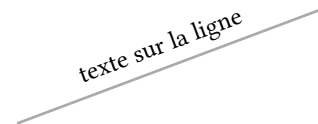


```
let p = ((0,0), (1.4,.9), (2.2,-.4), (.6,-1.1))
catmull(..p, close: true, fill: orange.lighten(70%), stroke:
orange)
```



23.4 content : texte le long d'une ligne, wrap, auto-scale

```
line((0,0), (4,1.5), name: "l", stroke: gray)
// angle: <coordonnée> aligne le texte sur la direction
content(("l.start", 50%, "l.end"), angle: "l.end",
  anchor: "south", padding: .12)[texte sur la ligne]
```



```
// wrap : applique une fonction au contenu
content((0,0), padding: .15, frame: "rect",
  wrap: text.with(fill: white, weight: "bold"),
  fill: blue)[Wrap]
// auto-scale : le contenu suit l'échelle du canvas
scope({
  scale(1.8)
  content((1.6,0), auto-scale: true)[Scalé]
})
```



Scalé

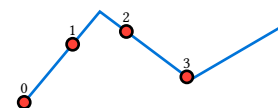
23.5 arc : update-position et ancrés

```
arc((0,0), start: 0deg, stop: 120deg, radius: 1, name: "a")
for (n, c) in (("arc-start", red), ("arc-end", blue),
  ("origin", green), ("chord-center", orange))
{
  circle("a." + n, radius: .07, fill: c)
}
content((2.6,.6), anchor: "west", text(6pt)[
  #text(fill: red)[arc-start] / #text(fill: blue)[arc-end]
\
  #text(fill: green)[origin] / #text(fill: orange)[chord-
center]])
```



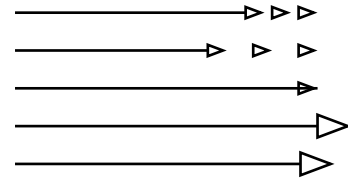
23.6 line : ancrés numérotés des sommets

```
line((0,0), (1,1.2), (2.2,.3), (3.4,1),
  name: "p", stroke: blue)
// chaque sommet est une ancre "0", "1", "2", ...
for i in range(4) {
  circle("p." + str(i), radius: .08, fill: red)
  content("p." + str(i), anchor: "south",
    padding: .12, text(6pt)[#i])
}
```



23.7 mark : offset, sep, shorten-to, transform-shape

```
// plusieurs marques, espacement contrôlé par sep
line((0,2), (4,2), mark: (end: ">", ">", ">"), sep: .1))
line((0,1.5), (4,1.5), mark: (end: ">", ">", ">"), sep: .35))
// offset décale la marque le long du chemin
line((0,1), (4,1), mark: (end: ">", offset: -.6))
// anchor : la marque est posée par sa base / son centre
line((0,.5), (4,.5), mark: (end: ">", anchor: "base", scale:
2))
line((0,0), (4,0), mark: (end: ">", anchor: "center", scale:
2))
```

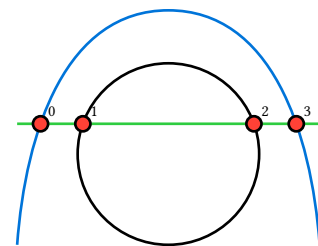


```
// shorten-to : jusqu'à quelle marque le trait est raccourci
line((0,1), (3,1), stroke: 3pt + blue,
  mark: (end: ">", ">"), shorten-to: 0, fill: blue))
line((0,0), (3,0), stroke: 3pt + red,
  mark: (end: ">", ">"), shorten-to: auto, fill: red))
```



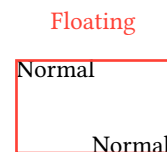
23.8 intersections : tri des points

```
intersections(name: "i", "x", {
  circle((0,0), radius: 1.2)
  hobby((-2,-1.2), (0,1.9), (2,-1.2), stroke: blue)
  line((-2,.4), (2,.4), stroke: green)
}, samples: 25, sort: cetz.sorting.points-by-angle)
// les points sont numérotés dans l'ordre de tri
for-each-anchor("x", n => {
  circle((0,0), radius: .1, fill: red)
  content((0,0), anchor: "south-west", padding: .1, text(6pt, n))
})
```



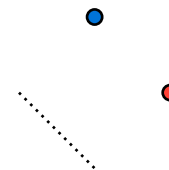
23.9 floating : ignorer un élément dans les bornes

```
group(name: "g", {
  content((1,0), [Normal])
  content((0,1), [Normal])
  floating(content((.5,1.6), text(fill: red)[Floating]))
})
// le rect ne tient pas compte de l'élément flottant
rect("g.north-west", "g.south-east", stroke: red)
```

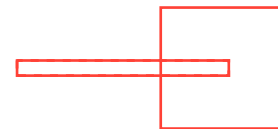


23.10 hide : calculé mais non dessiné

```
// l'élément caché reste nommable et ancrable
hide(circle((0,0), radius: 1, name: "h"))
circle("h.east", radius: .1, fill: red)
circle("h.north", radius: .1, fill: blue)
line("h.west", "h.south", stroke: (dash: "dotted"))
```

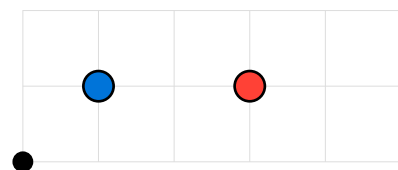


```
// bounds: true -> l'élément caché compte dans les bornes du
// canvas
// (comparez la taille des deux cadres pointillés)
hide(circle((0,0), radius: .8))
hide(circle((2.6,0), radius: .8), bounds: true)
rect-around((0,0), (2.6,0), padding: .1,
  stroke: (paint: red, dash: "dashed"))
```



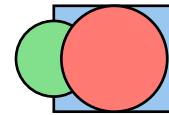
23.11 set-origin vs translate

```
grid((0,0), (5,2), stroke: gray.lighten(60%) + .3pt)
circle((0,0), radius: .12, fill: black)
// translate : cumulatif, déplace le repère
scope({ translate((1,1)); circle((0,0), radius: .2, fill:
blue) })
// set-origin : place l'origine sur une coordonnée résolue
scope({ set-origin((3,1)); circle((0,0), radius: .2, fill:
red) })
```



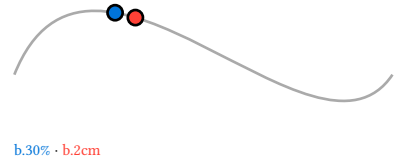
23.12 on-layer : ordre de peinture indépendant du code

```
// le disque rouge est écrit en premier mais peint au-dessus
on-layer(2, circle((.8,0), radius: .7, fill:
red.lighten(30%)))
rect((0,-.7), (1.6,.7), fill: blue.lighten(60%))
circle((0,0), radius: .5, fill: green.lighten(40%))
```



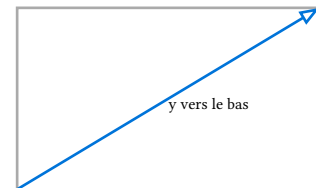
23.13 Ancres par distance ou par ratio

```
bezier((0,0), (5,0), (1,2.5), (4,-1.5), name: "b", stroke:
gray)
// ratio : proportion de la longueur
circle("b.30%", radius: .1, fill: blue)
// longueur : distance absolue depuis le départ.
// La forme "b.2cm" n'est pas parsée : il faut le
dictionnaire.
circle((name: "b", anchor: 2cm), radius: .1, fill: red)
content((0,-1), anchor: "west", text(6pt)[
#text(fill: blue)[b.30%] · #text(fill: red)[b.2cm]])
```



23.14 set-viewport : inverser un axe

```
rect((0,0), (4,2.4), stroke: gray)
// bounds inversés -> l'axe y pointe vers le bas (repère
écran)
set-viewport((0,0), (4,2.4), bounds: (100, -100))
line((0,0), (100,-100), stroke: blue, mark: (end: ">"))
content((50,-50), anchor: "north-west", text(6pt)[y vers le
bas])
```



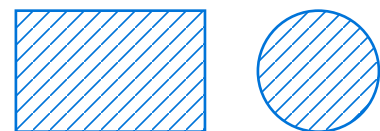
23.15 Dégradés suivant la forme

```
circle((0,0), radius: 1,
fill: gradient.radial(white, blue, focal-center: (35%,
35%)))
rect((2,-1), (4,1),
fill: gradient.linear(..color.map.rainbow, angle: 45deg),
stroke: none)
n-star((5.6,0), 5, radius: 1, inner-radius: 45%,
fill: gradient.conic(..color.map.plasma))
```



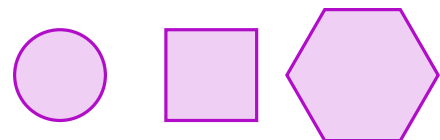
23.16 Motifs (tiling)

```
// attention : `line` est ici celui de cetz.draw,
// il faut donc qualifier celui de Typst avec `std.`
let hatch = tiling(size: (6pt, 6pt), {
place(std.line(start: (0pt, 6pt), end: (6pt, 0pt),
stroke: .6pt + blue))
})
rect((0,0), (2.5,1.6), fill: hatch, stroke: blue)
circle((4,.8), radius: .8, fill: hatch, stroke: blue)
```



23.17 Styles imbriqués et .. d'un dictionnaire

```
let theme = (stroke: (paint: purple, thickness: 1.2pt),
fill: purple.lighten(80%))
// on peut étaler un dictionnaire de style
circle((0,0), radius: .6, ..theme)
rect((1.4,-.6), (2.6,.6), ..theme)
// ... ou l'installer dans le contexte
set-style(..theme)
polygon((4,0), 6)
```

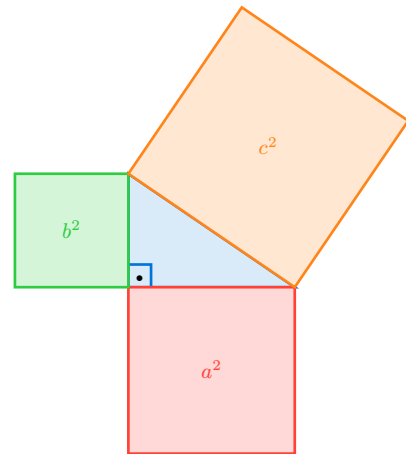


24 Galerie

Des figures complètes, plus longues, qui combinent plusieurs techniques. Elles servent de point de départ à copier-coller.

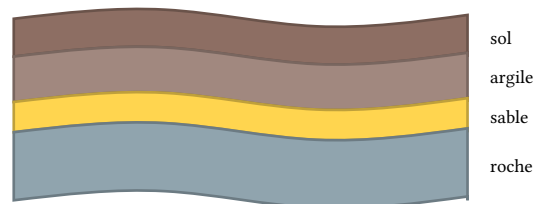
24.1 Le théorème de Pythagore

```
let (a, b) = (2.2, 1.5)
// triangle rectangle
line((0,0), (a,0), (0,b), close: true,
  fill: blue.lighten(85%), stroke: blue)
angle-lib.right-angle((0,0), (a,0), (0,b), radius: .3,
stroke: blue)
// carré sur chaque côté
line((0,0), (a,0), (a,-a), (0,-a), close: true,
  fill: red.lighten(80%), stroke: red)
content((a/2, -a/2), text(8pt, fill: red)[$a^2$])
line((0,0), (0,b), (-b,b), (-b,0), close: true,
  fill: green.lighten(80%), stroke: green)
content((-b/2, b/2), text(8pt, fill: green)[$b^2$])
// carré sur l'hypoténuse
let h = vector.sub((0,b,0), (a,0,0))
let n = (h.at(1), -h.at(0), 0)
line((a,0), (0,b), vector.add((0,b,0), n),
  vector.add((a,0,0), n), close: true,
  fill: orange.lighten(80%), stroke: orange)
content(vector.add(vector.lerp((a,0,0), (0,b,0), .5),
  vector.scale(n, .5)), text(8pt, fill: orange)
[$c^2$])
```



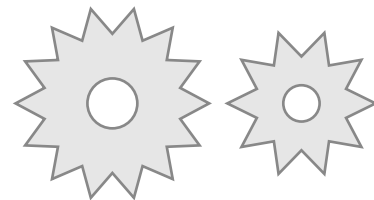
24.2 Coupe géologique / couches empilées

```
let layers = (
  (0.0, 0.5, rgb("#8d6e63"), "sol"),
  (0.5, 1.1, rgb("#a1887f"), "argile"),
  (1.1, 1.5, rgb("#ffd54f"), "sable"),
  (1.5, 2.4, rgb("#90a4ae"), "roche"),
)
for (y0, y1, c, nom) in layers {
  merge-path(close: true, fill: c, stroke:
c.darken(25%), {
    // surface ondulée en haut de couche
    hobby((0, -y0), (2, -y0 + .12), (4, -y0 - .1), (6,
-y0 + .05))
    line((6, -y0), (6, -y1))
    hobby((6, -y1 + .05), (4, -y1 - .1), (2, -y1
+ .12), (0, -y1))
    line((0, -y1), (0, -y0))
  })
  content((6.3, -(y0 + y1) / 2), anchor: "west",
text(7pt, nom))
}
```



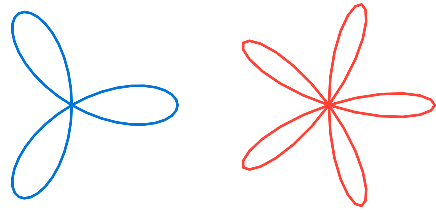
24.3 Engrenage paramétrique

```
let gear(pos, teeth: 12, r: 1, h: .18, name: none) = {
  let pts = ()
  let step = 360deg / (teeth * 2)
  for i in range(teeth * 2) {
    let a = step * i
    let rr = if calc.rem(i, 2) == 0 { r + h } else { r
- h }
    pts.push((pos.at(0) + calc.cos(a) * rr,
      pos.at(1) + calc.sin(a) * rr))
  }
  line(..pts, close: true, fill: gray.lighten(70%),
    stroke: gray.darken(20%), name: name)
  circle(pos, radius: r * .3, fill: white, stroke:
gray.darken(20%))
}
gear((0,0), teeth: 14, r: 1.1)
gear((2.5,0), teeth: 10, r: .8)
```



24.4 Rosace (courbe polaire)

```
// r = cos(k·θ) : rosace à k ou 2k pétales
for (k, c) in ((3, blue), (5, red)) {
  let pts = range(0, 361, step: 3).map(d => {
    let t = d * 1deg
    let r = calc.cos(k * t) * 1.4
    (calc.cos(t) * r + (if k == 3 { 0 } else { 3.4 }),
     calc.sin(t) * r)
  })
  line(..pts, close: true, stroke: c + .8pt)
}
```



24.5 Spirale d'Archimède et suite de Fibonacci

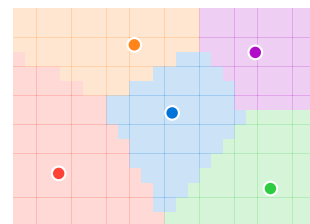
```
// spirale : r croît linéairement avec l'angle
let pts = range(0, 1081, step: 6).map(d => {
  let t = d * 1deg
  let r = d / 1080 * 1.8
  (calc.cos(t) * r, calc.sin(t) * r)
})
line(..pts, stroke: gradient.linear(blue, red) + 1.2pt)

// carrés de Fibonacci
let (x, y) = (2.6, -1.8)
let fib = (1, 1, 2, 3, 5)
let s = .3
for (i, f) in fib.enumerate() {
  rect((x, y), (x + f * s, y + f * s),
       stroke: teal, fill: teal.transparentize(90%))
  x += f * s
}
```



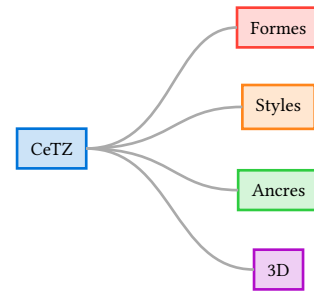
24.6 Diagramme de Voronoï (approché)

```
let sites = ((0.6, 0.7), (2.1, 1.5), (3.4, 0.5),
             (1.6, 2.4), (3.2, 2.3))
let cols = (red, blue, green, orange, purple)
// échantillonnage : chaque cellule est coloriée point
// par point
let n = 26
for i in range(n) {
  for j in range(int(n * 0.6)) {
    let p = (i / n * 4, j / (n * 0.6) * 3)
    let best = 0
    let bd = 1e9
    for (k, s) in sites.enumerate() {
      let d = vector.dist((p.at(0), p.at(1), 0),
                          (s.at(0), s.at(1), 0))
      if d < bd { bd = d; best = k }
    }
    rect(p, (rel: (4 / n, 3 / (n * 0.6))), stroke:
none,
        fill: cols.at(best).transparentize(80%))
  }
}
for (k, s) in sites.enumerate() {
  circle(s, radius: .09, fill: cols.at(k), stroke:
white + .8pt)
}
```



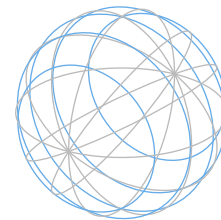
24.7 Carte mentale (arbre stylisé)

```
let node(pos, body, c, name) = {
  content(pos, padding: .18, frame: "rect", name: name,
    fill: c.lighten(80%), stroke: c, radius: .12,
    text(7pt, body))
}
node((0,0), [CeTZ], blue, "root")
let kids = ([Formes], 1.6), ([Styles], .55),
  ([Ancres], -.55), ([3D], -1.6)
for (i, (label, y)) in kids.enumerate() {
  node((3, y), label, (red, orange, green,
    purple).at(i),
    "k" + str(i))
  bezier("root.east", "k" + str(i) + ".west",
    (1.6, 0), (1.6, y), stroke: gray)
}
```



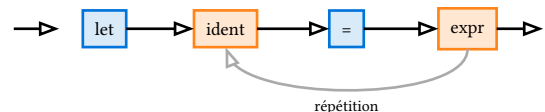
24.8 Sphère filaire (méridiens et parallèles)

```
ortho(x: 25deg, y: 30deg, {
  let r = 1.4
  // parallèles
  for k in range(-2, 3) {
    let z = k / 3 * r
    let rr = calc.sqrt(calc.max(r * r - z * z, 0))
    on-xy(z: z, circle((0,0), radius: rr,
      stroke: blue.lighten(40%)
+ .5pt))
  }
  // méridiens
  for i in range(6) {
    scope{
      rotate(z: 30deg * i)
      on-xz(y: 0, circle((0,0), radius: r,
        stroke: gray.lighten(20%)
+ .5pt))
    }
  }
})
```



24.9 Chemin de fer / diagramme syntaxique

```
let term(pos, body, name) = content(pos, padding: .16,
  frame: "rect", radius: .3, fill: blue.lighten(85%),
  stroke: blue, name: name, text(7pt, body))
let nonterm(pos, body, name) = content(pos,
  padding: .16,
  frame: "rect", fill: orange.lighten(80%),
  stroke: orange, name: name, text(7pt, body))
line((-0.6,0), (0,0), mark: (end: ">"))
term((0.6,0), [let], "a")
nonterm((2.2,0), [ident], "b")
term((3.8,0), [=], "c")
nonterm((5.4,0), [expr], "d")
for (p, q) in (("a","b"), ("b","c"), ("c","d")) {
  line(p + ".east", q + ".west", mark: (end: ">"))
}
line("d.east", (6.4,0), mark: (end: ">"))
// boucle de répétition
bezier("d.south", "b.south", (5.4,-1), (2.2,-1),
  stroke: gray, mark: (end: ">"))
content((3.8,-.95), anchor: "north", text(6pt)
[répétition])
```

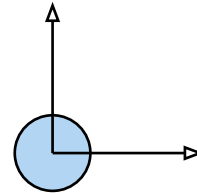


25 Couverture complémentaire

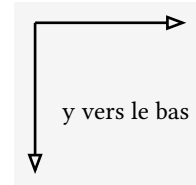
Les derniers recoins de l'API : options du canvas, styles complets des décorations, champs des nœuds d'arbre, et les fonctions de plot non encore montrées.

25.1 canvas : x, y, z, background, stroke

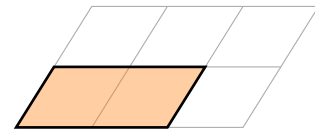
```
// x/y/z redéfinissent les vecteurs de base du repère
circle((0,0), radius: .5, fill: blue.lighten(70%))
line((0,0), (2,0), mark: (end: ">"))
line((0,0), (0,2), mark: (end: ">"))
```



```
// y: -1 inverse l'axe vertical (repère écran)
#cetz.canvas(y: -1, background: luma(96%), padding: .2, {
  import cetz.draw: *
  line((0,0), (2,0), mark: (end: ">"))
  line((0,0), (0,2), mark: (end: ">"))
  content((1.2,1.2))[y vers le bas]
})
```

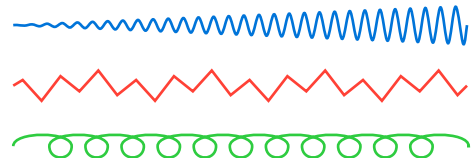


```
// vecteurs de base obliques -> repère « isométrique » 2D
#cetz.canvas(x: (1, 0), y: (.5, .8), {
  import cetz.draw: *
  grid((0,0), (3,2), stroke: gray + .4pt)
  rect((0,0), (2,1), fill: orange.transparentize(60%))
})
```

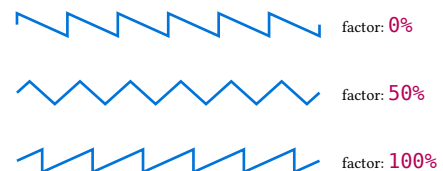


25.2 Décorations : le style au complet

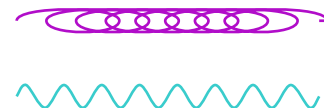
```
// amplitude peut être une fonction : elle reçoit un RATIO
// (0%..100%) et doit renvoyer un nombre
decorations.wave(line((0,1.6), (6,1.6)),
  amplitude: r => r / 100% * .5, segments: 30, stroke:
  blue)
// ... ou un tableau, utilisé cycliquement
decorations.zigzag(line((0,.8), (6,.8)),
  amplitude: (.15, .4, .25), segments: 12, stroke: red)
// segment-length fixe la taille au lieu du nombre
decorations.coil(line((0,0), (6,0)),
  amplitude: .3, segment-length: .5, stroke: green)
```



```
// factor : forme du zigzag et du créneau
for (i, f) in (0%, 50%, 100%).enumerate() {
  decorations.zigzag(line((0, -i * .9), (4, -i * .9)),
    amplitude: .3, segments: 6, factor: f, stroke: blue)
  content((4.3, -i * .9), anchor: "west", text(6pt)[factor:
  #f])
}
```

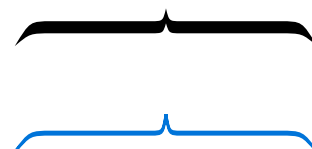


```
// coil : factor = dépassement (bouclage) ; wave : tension
decorations.coil(line((0,1), (4,1)), amplitude: .3,
  segments: 8, factor: 250%, stroke: purple)
decorations.wave(line((0,0), (4,0)), amplitude: .3,
  segments: 8, tension: 1.2, stroke: teal)
```



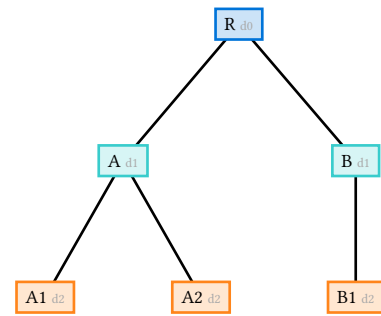
25.3 brace : accolade effilée

```
// taper + fill : accolade calligraphique
decorations.brace((0,0), (4,0), amplitude: .5,
  thickness: .08, pointiness: 40%,
  fill: black, stroke: none)
decorations.brace((0,-1.4), (4,-1.4), amplitude: .5,
  pointiness: 100%, stroke: blue)
```

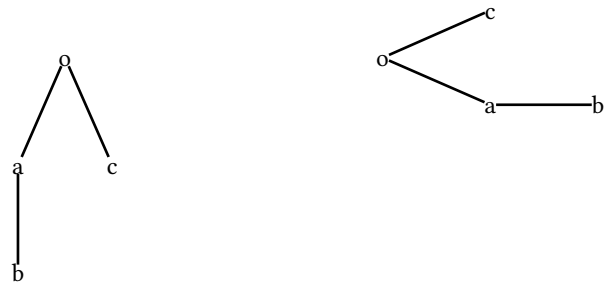


25.4 Arbres : champs du nœud et alignement

```
tree.tree(
  ([R], ([A], [A1], [A2]), ([B], [B1])),
  spread: 1.3, grow: 1.4,
  // le nœud expose .content .depth .n .children .name
  draw-node: node => {
    let c = (blue, teal, orange).at(calc.min(node.depth,
2))
    content((0,0), padding: .12, frame: "rect", radius: .1,
      fill: c.lighten(80%), stroke: c,
      text(7pt)[#node.content #text(5pt, fill: gray)
[d#node.depth]])
  })
```

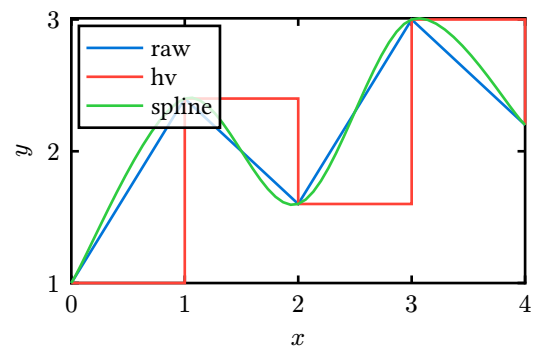


```
// direction et anchor : aligner l'arbre sur sa racine
for (i, d) in ("down", "right").enumerate() {
  scope({
    translate((i * 4.2, 0))
    tree.tree([([o], ([a], [b]), [c]), direction: d,
      grow: 1.2, spread: 1.1, anchor: "0")
  })
}
```



25.5 plot : types de ligne

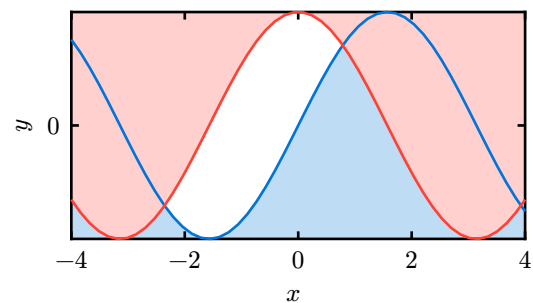
```
// #import "@preview/cetz-plot:0.1.4": plot
let d = ((0,1), (1,2.4), (2,1.6), (3,3), (4,2.2))
plot.plot(size: (6, 3.5), x-tick-step: 1, y-tick-step:
1,
  legend: "inner-north-west", {
    plot.add(d, line: "raw", label: "raw")
    plot.add(d, line: "hv", label: "hv",
      style: (stroke: red))
    plot.add(d, line: "spline", label: "spline",
      style: (stroke: green))
  })
```



Types disponibles : "raw" (défaut), "linear", "spline" (avec tension, samples), "vh", "hv", "hvh" (avec mid).

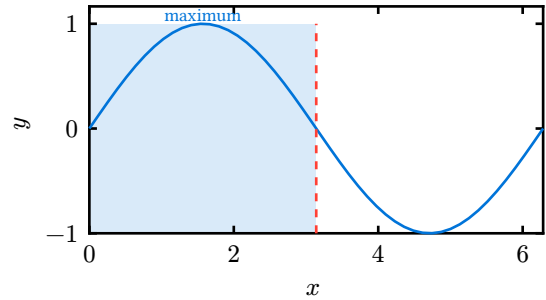
25.6 plot : hypographe, épigraphe, remplissage

```
// #import "@preview/cetz-plot:0.1.4": plot
plot.plot(size: (6, 3), x-tick-step: 2, y-tick-step: 1,
{
  plot.add(domain: (-4, 4), x => calc.sin(x),
    hypograph: true, style: (stroke: blue))
  plot.add(domain: (-4, 4), x => calc.cos(x),
    epigraph: true, style: (stroke: red))
})
```



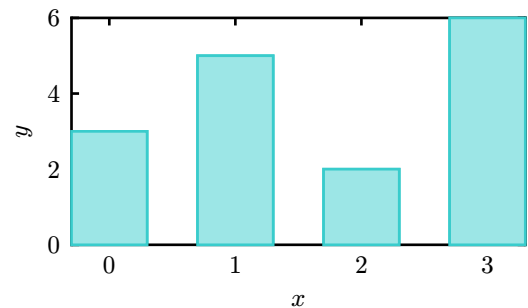
25.7 plot.annotate : dessiner dans les coordonnées du tracé

```
// #import "@preview/cetz-plot:0.1.4": plot
plot.plot(size: (6, 3), x-tick-step: 2, y-tick-step: 1,
{
  plot.add(domain: (0, 2 * calc.pi), calc.sin)
  plot.annotate(background: true, {
    rect((0, -1), (calc.pi, 1), fill:
      blue.transparentize(85%),
      stroke: none)
  })
  plot.annotate({
    content((calc.pi / 2, 1), anchor: "south",
      text(7pt, fill: blue)[maximum])
    line((calc.pi, -1), (calc.pi, 1),
      stroke: (paint: red, dash: "dashed"))
  })
})
```



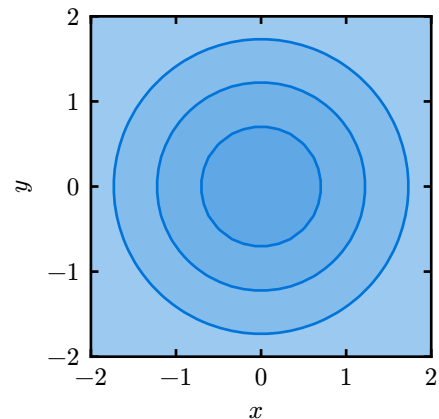
25.8 plot.add-bar : barres dans un tracé

```
// #import "@preview/cetz-plot:0.1.4": plot
plot.plot(size: (6, 3), x-tick-step: 1, y-tick-step: 2,
{
  plot.add-bar(((0, 3), (1, 5), (2, 2), (3, 6)),
    bar-width: .6, style: (fill:
      teal.lighten(50%),
      stroke: teal))
})
```



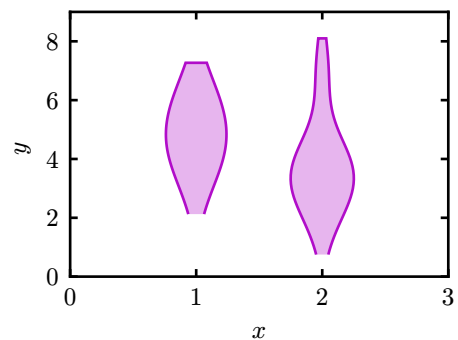
25.9 plot.add-contour : lignes de niveau

```
// #import "@preview/cetz-plot:0.1.4": plot
plot.plot(size: (4.5, 4.5), x-tick-step: 1, y-tick-
step: 1, {
  plot.add-contour(
    (x, y) => x * x + y * y,
    x-domain: (-2, 2), y-domain: (-2, 2),
    x-samples: 30, y-samples: 30,
    z: (.5, 1.5, 3), fill: true,
    style: (stroke: blue, fill:
      blue.transparentize(85%)))
})
```



25.10 plot.add-violin : distribution

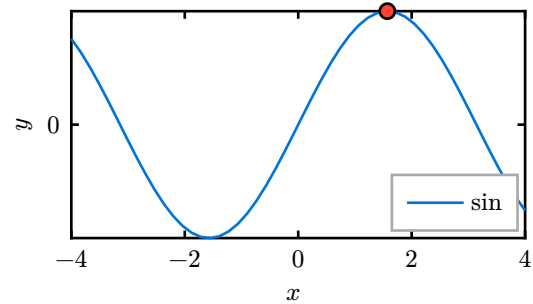
```
// #import "@preview/cetz-plot:0.1.4": plot
// chaque entrée est (x, <tableau de valeurs>) et les
// bornes
// des axes doivent être fixées à la main
plot.plot(size: (5, 3.5), x-tick-step: 1, y-tick-step:
2,
x-min: 0, x-max: 3, y-min: 0, y-max: 9, {
  plot.add-violin((
    (1, (3, 4, 4.5, 5, 6, 6.5, 5.5, 4.2)),
    (2, (2, 3, 3.5, 4, 7, 3.2, 3.8))),
    ), side: "both", bandwidth: .8,
    style: (fill: purple.transparentize(70%), stroke:
      purple))
})
```



add-violin en 0.1.4 Deux contraintes non documentées : les données sont des couples (x, (v1, v2, ...)) — pas une liste de points — et x-min/x-max/y-min/y-max sont **obligatoires**, sinon l'axe échoue avec Axis min and max must be set. side vaut "left", "right" (défaut) ou "both".

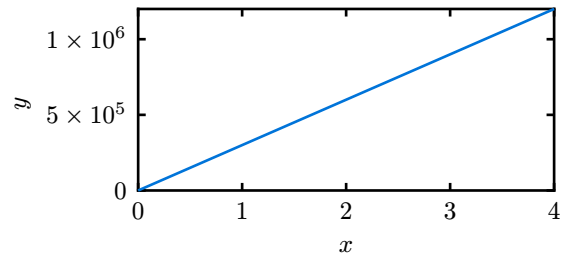
25.11 plot : légende et ancres personnalisées

```
// #import "@preview/cetz-plot:0.1.4": plot
plot.plot(size: (6, 3), x-tick-step: 2, y-tick-step: 1,
  legend: "inner-south-east",
  legend-style: (fill: white, stroke: gray,
padding: .2), {
  plot.add(domain: (-4, 4), calc.sin, label: $sin$)
  // une ancre nommée, utilisable après le plot
  plot.add-anchor("sommet", (calc.pi / 2, 1))
}, name: "p")
circle("p.sommet", radius: .1, fill: red)
```



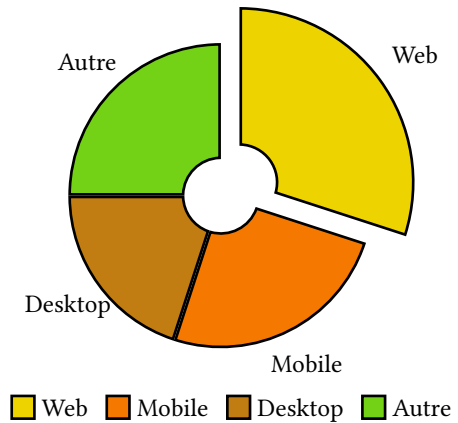
25.12 plot.formats : formats de graduation

```
// #import "@preview/cetz-plot:0.1.4": plot
plot.plot(size: (5.5, 2.4), x-tick-step: 1, y-tick-
step: 5e5,
  y-format: plot.formats.sci,
  x-format: plot.formats.decimal.with(digits: 1), {
  plot.add(domain: (0, 4), x => x * 3e5)
})
```



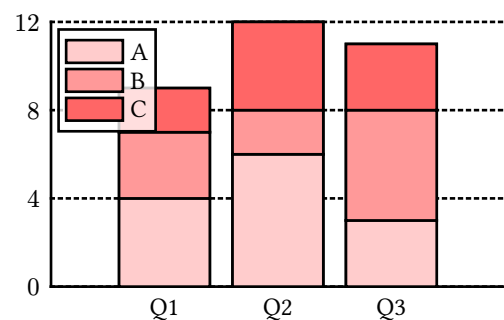
25.13 chart.piechart : étiquettes et éclatement

```
// #import "@preview/cetz-plot:0.1.4": chart
chart.piechart(
  ((30, [Web]), (25, [Mobile]), (20, [Desktop]), (25,
[Autre])),
  value-key: 0, label-key: 1,
  radius: 2, outset: (0, 0), outset-offset: .3,
  slice-style: cetz.palette.tango,
  inner-radius: .5)
```



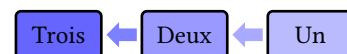
25.14 chart.columnchart : mode empilé

```
// #import "@preview/cetz-plot:0.1.4": chart
let data = (("Q1", 4, 3, 2), ("Q2", 6, 2, 4), ("Q3", 3,
5, 3))
chart.columnchart(data, mode: "stacked", value-key: (1,
2, 3),
  size: (6, 3.5), y-tick-step: 4,
  labels: ([A], [B], [C]), legend: "inner-north-west")
```

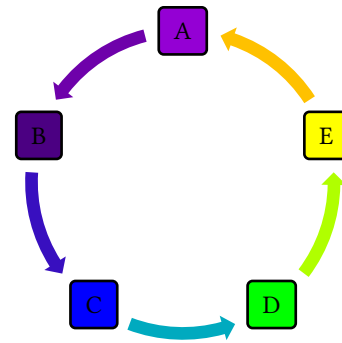


25.15 smartart : options de mise en forme

```
// #import "@preview/cetz-plot:0.1.4": smartart
smartart.process.basic([Un], [Deux], [Trois]),
  equal-width: true, dir: rtl,
  step-style: cetz.palette.blue)
```



```
// #import "@preview/cetz-plot:0.1.4": smartart
smartart.cycle.basic([A], [B], [C], [D], [E]),
  radius: 2, ccw: true, step-style:
  cetz.palette.rainbow)
```



26 Géométrie euclidienne

Ce chapitre montre que les constructions classiques — celles de [tkz-euclide](#) — s'obtiennent avec les **seules primitives de CeTZ**, sans une ligne de bibliothèque externe. Tout repose sur cinq mécanismes déjà vus :

Outil CeTZ	Ce qu'il donne en géométrie
circle-through(A, B, C)	le cercle circonscrit, et son centre par l'ancre .center
(bary: (A: 1, B: 1, C: 1))	tout point défini par un barycentre : centre de gravité, centre inscrit, exinscrits
(P, "_ _", A, B)	le projeté orthogonal de P sur (AB) : pied de hauteur, point de contact
intersections(...)	l'intersection de deux droites ou courbes : orthocentre, points d'un cercle
(A, 50%, B)	milieu et, plus généralement, tout point d'un segment

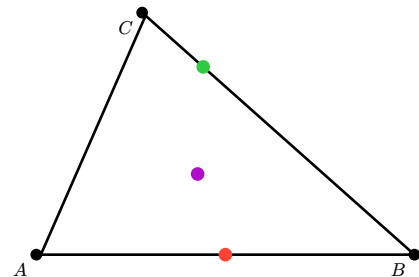
Nommer avant d'utiliser Les coordonnées bary, "_|_" et les ancres exigent des éléments **nommés**. On pose donc d'abord les points avec circle(..., name: "A"), puis on y fait référence par la chaîne "A". Une variable Typst ne suffit pas.

26.1 Les points de base

```
// on matérialise les sommets, ce qui les nomme
circle((0,0), radius: .06, fill: black, name: "A")
circle((5,0), radius: .06, fill: black, name: "B")
circle((1.4,3.2), radius: .06, fill: black, name: "C")
line("A", "B", "C", close: true)

// milieu : interpolation à 50 %
circle(("A", 50%, "B"), radius: .09, fill: red, stroke:
none)
// centre de gravité : barycentre équipondéré
circle((bary: (A: 1, B: 1, C: 1)), radius: .09, fill:
purple, stroke: none)
// pied de la hauteur issue de A : projection
circle(("A", "_|_", "B", "C"), radius: .09, fill:
green, stroke: none)

for (p, n) in (("A", $A$), ("B", $B$), ("C", $C$)) {
  content(p, anchor: "north-east", padding: .12,
text(7pt, n))
}
```



26.2 Cercle circonscrit et médiatrices

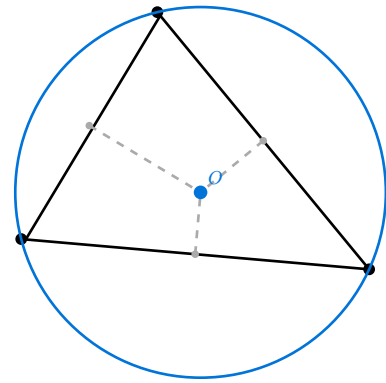
circle-through fait tout le travail ; son ancre .center donne O.

```

circle((0,0),      radius: .06, fill: black, name: "A")
circle((4.6,-.4),  radius: .06, fill: black, name: "B")
circle((1.8,3),    radius: .06, fill: black, name: "C")
line("A", "B", "C", close: true)
circle-through("A", "B", "C", name: "cc", stroke: blue
+ 1pt)

// une médiatrice = la droite joignant le milieu au
centre
for (p, q) in (("A","B"), ("B","C"), ("C","A")) {
  let m = (p, 50%, q)
  circle(m, radius: .05, fill: gray, stroke: none)
  line(m, "cc.center", stroke: (paint: gray, dash:
"dashed"))
}
circle("cc.center", radius: .09, fill: blue, stroke:
none)
content("cc.center", anchor: "south-west", padding: .1,
text(7pt, fill: blue, $0$))

```



26.3 Orthocentre par intersection des hauteurs

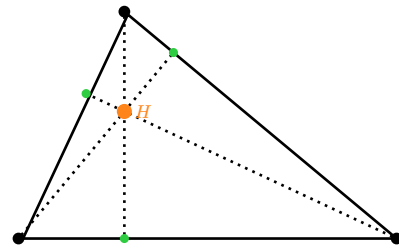
```

circle((0,0),      radius: .06, fill: black, name: "A")
circle((5,0),      radius: .06, fill: black, name: "B")
circle((1.4,3),    radius: .06, fill: black, name: "C")
line("A", "B", "C", close: true)

// `intersections` dessine les hauteurs et nomme leur
point commun
intersections(name: "x", "H", {
  line("A", ("A", "_|_", "B", "C"), stroke: (dash:
"dotted"))
  line("B", ("B", "_|_", "C", "A"), stroke: (dash:
"dotted"))
  line("C", ("C", "_|_", "A", "B"), stroke: (dash:
"dotted"))
})
circle("H.0", radius: .1, fill: orange, stroke: none)
content("H.0", anchor: "west", padding: .14, text(7pt,
fill: orange, $H$))

// les pieds des hauteurs
for (s, p, q) in (("A","B","C"), ("B","C","A"),
("C","A","B")) {
  circle((s, "_|_", p, q), radius: .06, fill: green,
stroke: none)
}

```



26.4 Cercle inscrit : un barycentre bien choisi

Le centre du cercle inscrit est le barycentre des sommets pondérés par les longueurs des côtés opposés. `get-ctx` donne accès aux coordonnées résolues pour les mesurer.

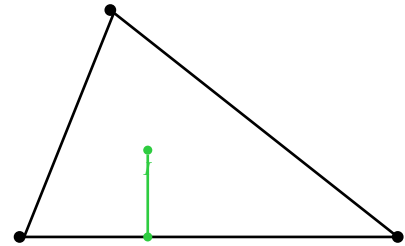
```

let (A, B, C) = ((0,0), (5,0), (1.2,3))
circle(A, radius: .06, fill: black, name: "A")
circle(B, radius: .06, fill: black, name: "B")
circle(C, radius: .06, fill: black, name: "C")
line("A", "B", "C", close: true)

// longueurs des côtés, avec les utilitaires vectoriels
// de CeTZ
let a = cetz.vector.dist(B, C)
let b = cetz.vector.dist(C, A)
let c = cetz.vector.dist(A, B)
let I = (bary: (A: a, B: b, C: c))

// le rayon est la distance de I à un côté : |I -
// projeté|
circle(I, radius: .06, fill: green, stroke: none, name:
"I")
intersections(name: "t", "T", {
  circle(I, radius: 3, stroke: none) // cercle
  // auxiliaire, invisible
  line("A", "B", stroke: none)
})
// plus simple : le point de contact est le projeté de
// I sur (AB)
line("I", ("I", "_|_", "A", "B"), stroke: green)
circle(("I", "_|_", "A", "B"), radius: .06, fill:
green, stroke: none)
content("I", anchor: "north", padding: .16, text(7pt,
fill: green, $I$))

```



Pour tracer le cercle lui-même, on mesure le rayon dans get-ctx, où les coordonnées sont déjà résolues en vecteurs :

```

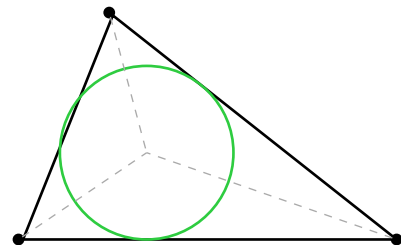
let (A, B, C) = ((0,0), (5,0), (1.2,3))
circle(A, radius: .06, fill: black, name: "A")
circle(B, radius: .06, fill: black, name: "B")
circle(C, radius: .06, fill: black, name: "C")
line("A", "B", "C", close: true)

let (a, b, c) = (cetz.vector.dist(B, C),
  cetz.vector.dist(C, A),
  cetz.vector.dist(A, B))

let s = (a + b + c) / 2
// aire par la formule de Héron, puis r = aire / demi-
// périmètre
let aire = calc.sqrt(s * (s - a) * (s - b) * (s - c))
circle((bary: (A: a, B: b, C: c)), radius: aire / s,
stroke: green + 1pt)

// les bissectrices concourent au centre
for s in ("A", "B", "C") {
  line(s, (bary: (A: a, B: b, C: c)),
    stroke: (paint: gray, dash: "dashed",
    thickness: .5pt))
}

```



26.5 La droite d'Euler

O, G et H sont alignés. On les obtient par trois mécanismes différents.

```

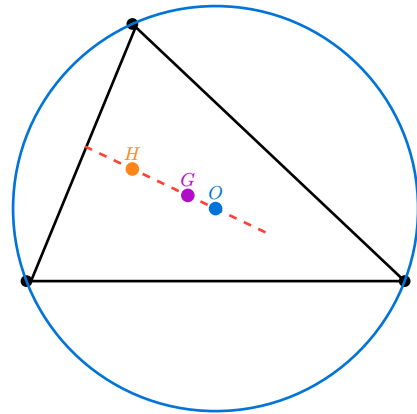
circle((0,0),      radius: .06, fill: black, name: "A")
circle((5,0),      radius: .06, fill: black, name: "B")
circle((1.4,3.4),  radius: .06, fill: black, name: "C")
line("A", "B", "C", close: true)

circle-through("A", "B", "C", name: "cc", stroke: blue)
intersections(name: "x", "H", {
  line("A", ("A", "_|_", "B", "C"), stroke: none)
  line("B", ("B", "_|_", "C", "A"), stroke: none)
})

// la droite passe par O et H ; on la prolonge par
// interpolation
// au-delà des extrémités : un ratio < 0 % ou > 100 %
// extrapole
line(("cc.center", -60%, "H.0"), ("cc.center", 160%,
"H.0"),
  stroke: (paint: red, dash: "dashed"))

for (p, c, n) in (("cc.center", blue, $0$), ("H.0",
orange, $H$),
  ((bary: (A: 1, B: 1, C: 1)), purple,
  $G$)) {
  circle(p, radius: .09, fill: c, stroke: none)
  content(p, anchor: "south", padding: .12, text(7pt,
fill: c, n))
}

```



Extrapoler une droite L'interpolation (A, t, B) accepte des ratios hors de [0%, 100%] : -60% recule avant A, 160% dépasse B. C'est le moyen le plus court de prolonger une droite définie par deux points, sans calcul de vecteur.

26.6 Le cercle des neuf points

Son centre est le milieu de [OH] et son rayon la moitié de celui du cercle circonscrit — deux valeurs qu'on lit sur les objets déjà construits.

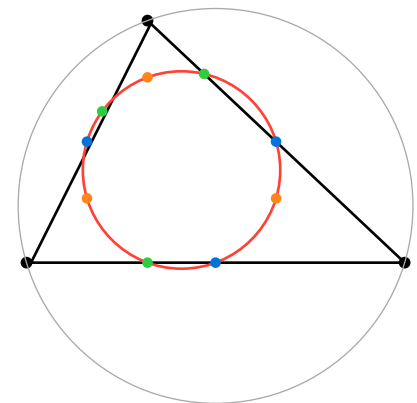
```

circle((0,0),      radius: .06, fill: black, name: "A")
circle((5,0),      radius: .06, fill: black, name: "B")
circle((1.6,3.2),  radius: .06, fill: black, name: "C")
line("A", "B", "C", close: true)
circle-through("A", "B", "C", name: "cc", stroke: gray
+ .5pt)
intersections(name: "x", "H", {
  line("A", ("A", "_|_", "B", "C"), stroke: none)
  line("B", ("B", "_|_", "C", "A"), stroke: none)
})

// rayon = distance centre → sommet, mesurée dans get-
// ctx
get-ctx(ctx => {
  let (_, o, a) = cetz.coordinate.resolve(ctx,
"cc.center", "A")
  circle(("cc.center", 50%, "H.0"),
    radius: cetz.vector.dist(o, a) / 2, stroke:
red + 1pt)
})

// les neuf points : 3 milieux, 3 pieds, 3 milieux des
// [HS]
for (p, q) in (("A", "B"), ("B", "C"), ("C", "A")) {
  circle((p, 50%, q), radius: .07, fill: blue, stroke:
none)
}
for (s, p, q) in (("A", "B", "C"), ("B", "C", "A"),
("C", "A", "B")) {
  circle((s, "_|_", p, q), radius: .07, fill: green,
stroke: none)
}
for s in ("A", "B", "C") {
  circle(("H.0", 50%, s), radius: .07, fill: orange,
stroke: none)
}

```



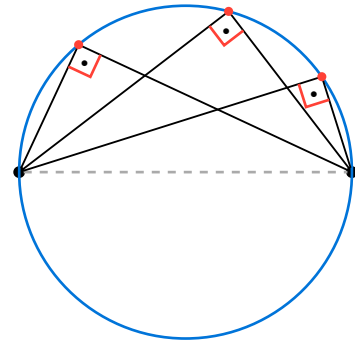
Mesurer une longueur `cetz.coordinate.resolve(ctx, ..)` transforme des coordonnées de tout système en vecteurs, dans un `get - ctx`. C'est la porte d'entrée pour tout calcul métrique — longueur, angle — à partir d'éléments nommés.

26.7 Théorème de Thalès

Tout point du cercle de diamètre [AB] voit ce diamètre sous un angle droit.

```
circle((0,0), radius: .06, fill: black, name: "A")
circle((4.4,0), radius: .06, fill: black, name: "B")
// le cercle de diamètre [AB] : centre au milieu,
// passant par A
circle(("A", 50%, "B"), radius: 2.2, stroke: blue,
name: "c")
line("A", "B", stroke: (paint: gray, dash: "dashed"))

// des points du cercle par ancre angulaire
for a in (35deg, 75deg, 130deg) {
  let M = "c." + repr(a)
  line("A", M, "B", stroke: .7pt)
  angle-lib.right-angle(M, "A", "B", radius: .32,
stroke: red)
  circle(M, radius: .06, fill: red, stroke: none)
}
```



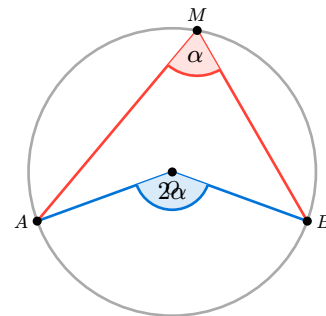
26.8 Théorème de l'angle inscrit

L'angle au centre vaut le double de l'angle inscrit sur le même arc. Les ancres angulaires d'un cercle donnent les points sans aucun calcul.

```
circle((0,0), radius: 1.9, stroke: gray, name: "c")
line("c.200deg", "c.center", "c.340deg", stroke: blue +
1pt)
line("c.200deg", "c.80deg", "c.340deg", stroke: red +
1pt)

angle-lib.angle("c.center", "c.200deg", "c.340deg",
radius: .5,
label: $2\alpha$, stroke: blue, fill:
blue.lighten(85%))
angle-lib.angle("c.80deg", "c.200deg", "c.340deg",
radius: .6,
label: $\alpha$, stroke: red, fill: red.lighten(85%))

for (p, n, an) in (("c.200deg", $A$, "east"),
("c.340deg", $B$, "west"),
("c.80deg", $M$, "south"),
("c.center", $O$, "north")) {
  circle(p, radius: .06, fill: black, stroke: none)
  content(p, anchor: an, padding: .12, text(7pt, n))
}
```

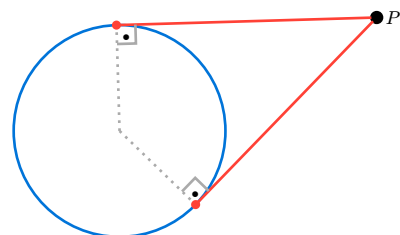


26.9 Tangentes à un cercle

La coordonnée tangente est native : elle donne le point de contact.

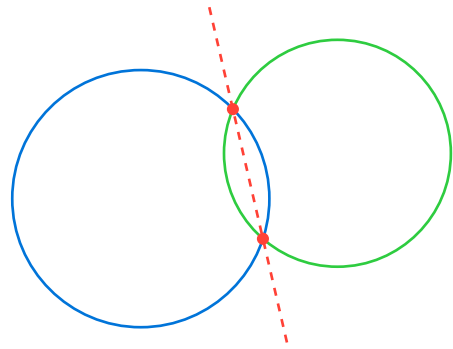
```
circle((0,0), radius: 1.4, stroke: blue, name: "c")
circle((3.4,1.5), radius: .07, fill: black, name: "P")

for s in (1, 2) {
  let T = (element: "c", point: "P", solution: s)
  line("P", T, stroke: red)
  line("c.center", T, stroke: (paint: gray, dash:
"dotted"))
  circle(T, radius: .06, fill: red, stroke: none)
  // le rayon est perpendiculaire à la tangente au
  // point de contact
  angle-lib.right-angle(T, "c.center", "P",
radius: .25, stroke: gray)
}
content("P", anchor: "west", padding: .12, text(7pt,
$P$))
```



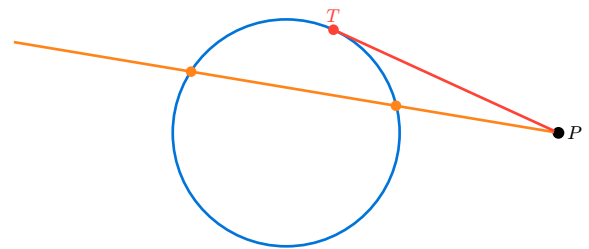
26.10 Deux cercles : points communs et axe radical

```
intersections(name: "i", "X", {
  circle((0,0), radius: 1.7, stroke: blue, name: "c1")
  circle((2.6,.6), radius: 1.5, stroke: green, name: "c2")
})
// l'axe radical est la droite passant par les deux
// intersections
line(("X.0", -80%, "X.1"), ("X.0", 180%, "X.1"),
  stroke: (paint: red, dash: "dashed"))
for n in ("X.0", "X.1") {
  circle(n, radius: .08, fill: red, stroke: none)
}
```



26.11 Puissance d'un point

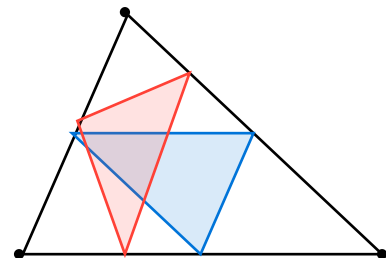
```
circle((0,0), radius: 1.5, stroke: blue, name: "c")
circle((3.6,0), radius: .06, fill: black, name: "P")
// une sécante issue de P coupe le cercle en deux
// points
intersections(name: "i", "S", {
  circle((0,0), radius: 1.5, stroke: none)
  line("P", (-3.6, 1.2), stroke: orange)
})
for n in ("S.0", "S.1") { circle(n, radius: .07, fill:
orange, stroke: none) }
// la tangente donne  $PT^2 = PA \cdot PB$ 
let T = (element: "c", point: "P", solution: 1)
line("P", T, stroke: red)
circle(T, radius: .07, fill: red, stroke: none)
content("P", anchor: "west", padding: .12, text(7pt,
$P$))
content(T, anchor: "south", padding: .1, text(7pt,
fill: red, $T$))
```



26.12 Triangle médian et triangle orthique

```
circle((0,0), radius: .05, fill: black, name: "A")
circle((4.8,0), radius: .05, fill: black, name: "B")
circle((1.4,3.2), radius: .05, fill: black, name: "C")
line("A", "B", "C", close: true, stroke: 1pt)

// médian : les trois milieux
line(("A", 50%, "B"), ("B", 50%, "C"), ("C", 50%, "A"),
close: true,
  fill: blue.transparentize(85%), stroke: blue)
// orthique : les trois pieds des hauteurs
line(("A", "_|_", "B", "C"), ("B", "_|_", "C", "A"),
  ("C", "_|_", "A", "B"), close: true,
  fill: red.transparentize(85%), stroke: red)
```



26.13 Droite de Simson

Les projetés d'un point du cercle circonscrit sur les trois côtés sont alignés. Pour tracer la droite entière il faut relier les deux projetés **extrêmes** : selon la position de M, le projeté « du milieu » n'est pas forcément celui qu'on croit — ici c'est celui sur (AB).

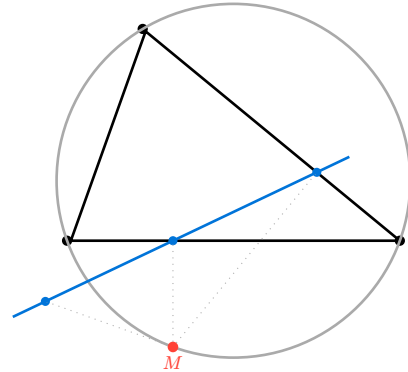
```

circle((0,0), radius: .05, fill: black, name: "A")
circle((4.4,0), radius: .05, fill: black, name: "B")
circle((1,2.8), radius: .05, fill: black, name: "C")
line("A", "B", "C", close: true)
circle-through("A", "B", "C", name: "cc", stroke: gray)

// un point du cercle, par son ancre angulaire
circle("cc.250deg", radius: .07, fill: red, stroke:
none, name: "M")
content("M", anchor: "north", padding: .12, text(7pt,
fill: red, $M$))

// les trois projetés
for (p, q) in (("A","B"), ("B","C"), ("C","A")) {
  let f = ("M", "_|_", p, q)
  circle(f, radius: .06, fill: blue, stroke: none)
  line("M", f, stroke: (paint: gray, dash: "dotted",
thickness: .4pt))
}
// on relie les deux projetés EXTRÊMES – ceux sur (BC)
et (CA) –
// en débordant un peu : le projeté sur (AB) tombe
entre les deux
line(((("M", "_|_", "B", "C"), -12%, ("M", "_|_", "C",
"A")),
(("M", "_|_", "B", "C"), 112%, ("M", "_|_", "C",
"A"))),
stroke: blue + 1pt)

```



26.14 Théorème de Ptolémée

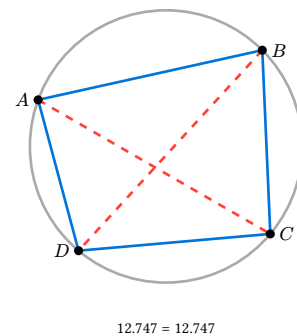
Pour un quadrilatère inscriptible, $AC \cdot BD = AB \cdot CD + BC \cdot AD$.

```

circle((0,0), radius: 1.8, stroke: gray, name: "c")
let (A, B, C, D) = ("c.160deg", "c.45deg", "c.-40deg",
"c.230deg")
line(A, B, C, D, close: true, stroke: blue + 1pt)
line(A, C, stroke: (paint: red, dash: "dashed"))
line(B, D, stroke: (paint: red, dash: "dashed"))

// vérification numérique dans get-ctx
get-ctx(ctx => {
  let (_, a, b, c, d) = cetz.coordinate.resolve(ctx, A,
B, C, D)
  let dist = cetz.vector.dist
  let g = dist(a, c) * dist(b, d)
  let h = dist(a, b) * dist(c, d) + dist(b, c) *
dist(a, d)
  content((0, -2.4), text(6pt)[
    #calc.round(g, digits: 3) = #calc.round(h, digits:
3)])
})
for (p, n, an) in ((A, $A$, "east"), (B, $B$, "west"),
(C, $C$, "west"), (D, $D$, "east"))
{
  circle(p, radius: .06, fill: black, stroke: none)
  content(p, anchor: an, padding: .12, text(7pt, n))
}

```



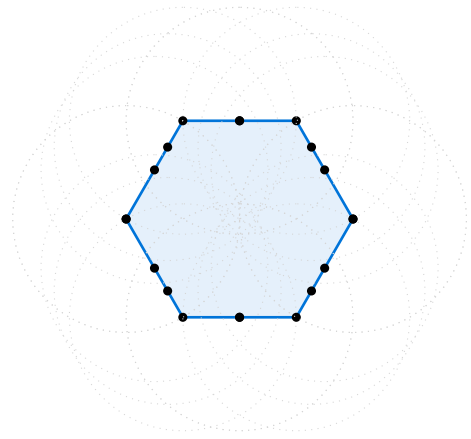
26.15 Polygones réguliers et pentagramme

polygone et n-star couvrent l'essentiel ; les ancres du polygone donnent ses sommets.

```

polygon((0,0), 6, radius: 1.5, stroke: blue + 1pt,
        fill: blue.transparentize(90%), name: "hex")
// report du rayon : les six cercles de la construction
// au compas
for-each-anchor("hex", exclude: ("center", "default"),
n => {
  circle(), radius: 1.5, stroke: (paint:
gray.lighten(50%),
                                thickness: .4pt,
dash: "dotted"))
  circle(), radius: .06, fill: black, stroke: none)
})

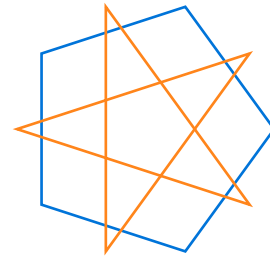
```



```

polygon((0,0), 5, radius: 1.7, stroke: blue + 1pt,
name: "p")
// les diagonales d'un pentagone forment le pentagramme
n-star((0,0), 5, radius: 1.7, inner-radius: 38.2%,
stroke: orange, show-inner: true)

```



26.16 Théorème de Napoléon

Les centres des triangles équilatéraux construits sur les côtés forment un triangle équilatéral. Le sommet extérieur s'obtient par la coordonnée d'interpolation **avec angle** : $(p, 100\%, -60\deg, q)$ place un point à la distance pq de p , dans la direction tournée de -60° .

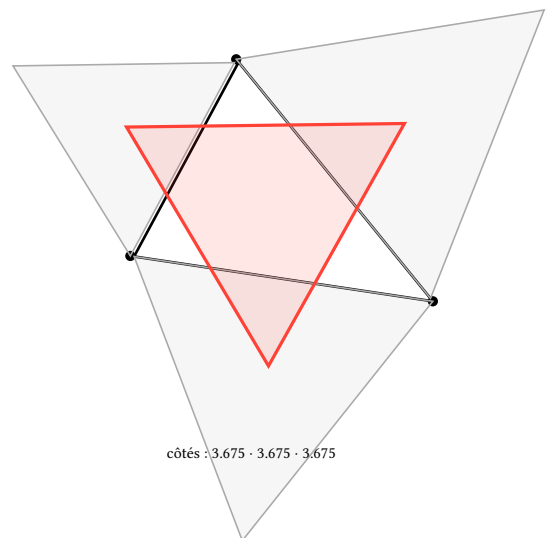
```

circle((0,0), radius: .05, fill: black, name: "A")
circle((4,-.6), radius: .05, fill: black, name: "B")
circle((1.4,2.6), radius: .05, fill: black, name: "C")
line("A", "B", "C", close: true, stroke: 1pt)

for (i, (p, q)) in (("A","B"), ("B","C"),
("C","A")).enumerate() {
  let S = (p, 100%, -60deg, q)
  line(p, q, S, close: true, stroke: gray + .6pt,
fill: gray.transparentize(90%))
  circle(S, radius: .04, stroke: none, name: "S" +
str(i))
}
// les centres, par barycentre des trois sommets
line((bary: (A: 1, B: 1, S0: 1)),
(bary: (B: 1, C: 1, S1: 1)),
(bary: (C: 1, A: 1, S2: 1)),
close: true, stroke: red + 1.2pt, fill:
red.transparentize(88%))

get-ctx(ctx => {
  let (_, ..c) = cetz.coordinate.resolve(ctx,
(bary: (A: 1, B: 1, S0: 1)),
(bary: (B: 1, C: 1, S1: 1)),
(bary: (C: 1, A: 1, S2: 1)))
  content((1.6, -2.6), text(6pt)[côtés :
#range(3).map(i =>
str(calc.round(cetz.vector.dist(c.at(i),
c.at(calc.rem(i + 1, 3))),
digits: 3))).join(" · ")]])
})

```

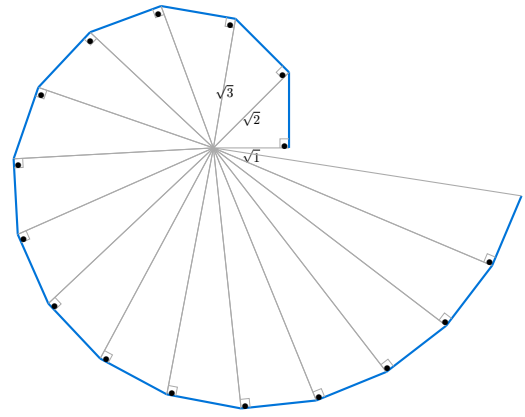


26.17 Spirale de Théodore

Chaque triangle rectangle ajoute une unité au carré de l'hypoténuse : les rayons successifs valent $\sqrt{1}, \sqrt{2}, \sqrt{3}, \dots$. Ici tout se fait par transformations, sans une seule coordonnée calculée à la main.

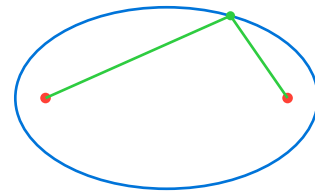
```
// à chaque étape on tourne du bon angle, puis on
rallonge
let r = 1.0
for i in range(1, 17) {
  let nr = calc.sqrt(i + 1)
  line((0,0), (r, 0), stroke: gray + .4pt)
  line((r, 0), (r, 1), stroke: blue + .8pt)
  line((0,0), (r, 1), stroke: gray + .4pt)
  angle-lib.right-angle((r, 0), (0,0), (r, 1),
    radius: .12,
    stroke: gray + .3pt)

  if i <= 3 {
    content((r / 2, 0), anchor: "north", padding: .06,
      text(5pt)[$sqrt(#str(i))$])
  }
  // on pivote pour que la nouvelle hypoténuse devienne
l'axe
  rotate(calc.atan2(r, 1.0))
  r = nr
}
```

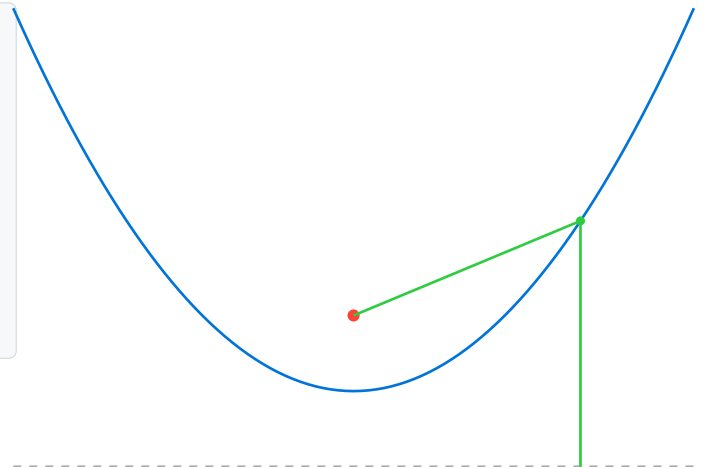


26.18 Coniques : ellipse, parabole, hyperbole

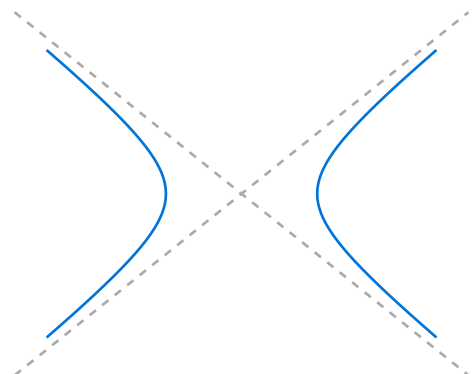
```
// ellipse : circle à rayon (rx, ry) ; les foyers par
calcul
let (a, b) = (2, 1.2)
circle((0,0), radius: (a, b), stroke: blue + 1pt)
let c = calc.sqrt(a * a - b * b)
for x in (-c, c) {
  circle((x, 0), radius: .07, fill: red, stroke: none)
}
// un point et ses rayons vecteurs
let M = (calc.cos(65deg) * a, calc.sin(65deg) * b)
line((-c, 0), M, (c, 0), stroke: green)
circle(M, radius: .06, fill: green, stroke: none)
```



```
// parabole y = x^2/4 : foyer (0,1), directrice y = -1
let pts = range(-45, 46).map(i => (i / 10, calc.pow(i /
10, 2) / 4))
line(..pts, stroke: blue + 1pt)
line((-4.5, -1), (4.5, -1), stroke: (paint: gray, dash:
"dashed"))
circle((0, 1), radius: .08, fill: red, stroke: none)
// la distance au foyer égale la distance à la
directrice
let P = (3, 9 / 4)
line((0,1), P, stroke: green)
line(P, (3, -1), stroke: green)
circle(P, radius: .06, fill: green, stroke: none)
```



```
// hyperbole x^2/a^2 - y^2/b^2 = 1, tracée par branches
let (a, b) = (1, .8)
for s in (-1, 1) {
  let pts = range(0, 41).map(i => {
    let t = i / 20 - 1
    (s * a * calc.cosh(t * 1.6), b * calc.sinh(t *
1.6))
  })
  line(..pts, stroke: blue + 1pt)
}
// les asymptotes
for s in (-1, 1) {
  line((-3, s * 3 * b / a), (3, -s * 3 * b / a),
    stroke: (paint: gray, dash: "dashed"))
}
```



27 Aide-mémoire

Squelette

```
#import "@preview/cetz:0.5.2"
#cezt.canvas(length: 1cm, {
  import cetz.draw: *
  ...
})
```

Coordonnées

```
(1, 2) (1, 2, 3) (x: 1, y: 2)
(30deg, 2) (angle: 30deg, radius: 2)
"elem" "elem.north"
(rel: (1, 0)) (rel: (1,0), to: "a")
(a, 50%, b) (a, 1.5, 30deg, b)
(a, "-|", b) (a, "|-", b)
(p, "-|_", a, b)
(bary: (a: 1, b: 2))
(element: "c", point: p, solution: 1)
() // point précédent
```

Formes

```
line(a, b, .., close: false)
rect(a, b, radius: 0)
circle(c, radius: 1)
circle-through(a, b, c)
arc(p, start: , stop: , delta: , mode: )
arc-through(a, b, c)
polygon(o, n, angle: 0deg)
n-star(o, n, inner-radius: 50%)
grid(from, to, step: 1)
content(p, [..], angle: , anchor: )
bezier(s, e, c1, [c2])
bezier-through(s, m, e)
catmull(..pts, tension: .5)
hobby(..pts, omega: (0,0))
merge-path({ .. }, close: false)
compound-path({ .. }, fill-rule: )
svg-path(("m", (0,0)), ("l", (1,1)))
rect-around("a", "b", padding: 0)
mark(from, to, symbol: ">")
```

Booléens

```
boolean(a, b, op: "union")
// "union" "intersection"
// "difference" "xor"
```

Extension

```
register-mark("n", style => {
  anchor("tip", (0,0))
  anchor("base", (style.length, 0))
  ..
}, mnemonic: "N")
register-coordinate-resolver(
  (ctx, c) => c)
```

Style

```
set-style(fill: , stroke: , radius: )
set-style(circle: (fill: red))
fill(red) stroke(2pt + blue)
// clés : fill fill-rule stroke radius
// padding mark shorten
```

Marques

```
mark: (end: ">")
mark: (start: ">", end: ">")
mark: (symbol: ">") // 2 bouts
mark: (end: (">", ">"))
mark: (end: ">", pos: 50%, scale: 2,
  fill: red, harpoon: true,
  flip: true, slant: 50%,
  anchor: "tip")
// > < <> [] ] [ | o + x * )> >> )
```

Ancres

```
"e.north" "e.center" "e.45deg"
"e.start" "e.mid" "e.end" "e.25%"
anchor("nom", (x, y))
copy-anchors("elem")
for-each-anchor("e", n => { .. })
```

Groupes & calques

```
group(name: "g", { .. })
scope({ .. })
hide(..) floating(..)
on-layer(1, { .. })
intersections("i", { .. })
get-ctx(ctx => { .. })
```

Transformations

```
translate((1, 2)) translate(x: 1)
rotate(30deg, origin: (0,0))
scale(2) scale(x: 2, y: .5)
set-origin((1,1)) move-to((2,0))
set-viewport(a, b, bounds: (10,10))
transform(mat) set-transform(mat)
```

3D

```
ortho({ on-xy({ .. }) })
perspective(distance: auto, { .. })
on-xy(z: 0, ..) on-xz(y: 0, ..)
on-zy(x: 0, ..)
```

Bibliothèques

```
cetz.tree.tree((root, c1, c2))
cetz.angle.angle(o, a, b, label: )
cetz.angle.right-angle(o, a, b)
cetz.decorations.zigzag / coil /
  wave / square / brace / flat-brace
cetz.palette.new(colors: (..))
```

Vecteurs / matrices

```
vector.add/sub/scale/div/neg
vector.len/dist/norm/lerp
vector.dot/cross/angle/angle2
matrix.ident(4) matrix.mul-mat(a,b)
matrix.transform-translate/scale
matrix.transform-rotate-x/y/z/xyz
matrix.inverse(m)
```

Géométrie (CeTZ pur)

```
// nommer d'abord : circle(p, name: "A")
circle-through("A","B","C", name: "cc")
"cc.center" // circonscrit
(bary: (A: 1, B: 1, C: 1)) // gravité
(bary: (A: a, B: b, C: c)) // inscrit
("A", 50%, "B") // milieu
("A", -60%, "B") // extrapole avant A
("A", 160%, "B") // ... et après B
("A", 100%, -60deg, "B") // + rotation
("P", "-|_", "A", "B") // projeté
("A", "-|", "B") ("A", "|-", "B")
(element: "c", point: "P", solution: 1)
intersections(name: "x", "H", { .. })
"c.45deg" // point du cercle
get-ctx(ctx => { // mesurer
  cetz.coordinate.resolve(ctx, "A", "B") })
```

Écosystème

```
// @preview/cetz-plot:0.1.4
plot.plot(size: (6,4), { plot.add(..) })
plot.add(domain: (a,b), x => y)
plot.add-hline/-vline/-fill-between
plot.add-errorbar(pt, y-error: .3)
plot.formats.sci / fraction /
  multiple-of / decimal
chart.columnchart / bargraph
chart.piechart / radarchart
```

```
chart.boxwhisker
smartart.process.basic / chevron
smartart.cycle.basic
// @preview/cetz-venn:0.2.0
venn.venn2(a-fill: red)   venn.venn3(..)
```